TECNOLÓGICO DE MONTERREY

Escuela de Ingeniería y Ciencias · Departamento de Ciencias Computacionales

PROYECTO INTEGRADOR · CORRIDAS V2 Y V3

Corrección y re-validación de las Fases 0 a 3, y comparación de políticas gaussiana y Beta en REINFORCE — bitácora razonada paso a paso

MSAPSO fiel, simulador compilado y comparación justa frente a agentes de aprendizaje por refuerzo (PPO) y REINFORCE con política gaussiana y Beta

Documento preparado para el sponsor del proyecto, en respuesta a las observaciones sobre la corrida V1 y a la petición de probar REINFORCE con distribución Beta (V3, pasos 13 y 14). Incluye el razonamiento de cada paso, los resultados con estadística entre semillas, las respuestas punto por punto, las limitaciones declaradas y el código clave.

Sponsor: Prof. Gerardo Jesús Camacho González, PhD. · Monterrey, Nuevo León, México · Octubre de 2026

Índice

Resumen General y recomendaciones 3

1. Resumen ejecutivo 5

2. Respuesta punto por punto a las observaciones del profesor 7

3. Qué estaba mal en la V1 y cómo se corrigió 9

4. Paso a paso de las corridas V2 y V3 11

- Paso 1 — Simulador V2 (mismo modelo físico, núcleo compilado) y validación contra la V1 13
- Paso 2 — MSAPSO fiel a la Fig. 6 y primera compuerta de validación 14
- Paso 3 — Ablación, diagnóstico de la dinámica del convertidor y decisión del régimen de tiempo 16
- Paso 4 — Entorno de RL V2, arnés común de evaluación y primera prueba de PPO 18
- Paso 5 — Rama REINFORCE (propuesta del profesor): implementación y malla en etapas 20
- Paso 6 — Primer PPO formal (semilla 0) y qué aprendió el agente 22
- Paso 7 — Control sin RL con selección en dos etapas (sin mirar los conjuntos de prueba) 24
- Paso 8 — PPO con 3 semillas y dos regímenes de tiempo (resultado principal) 25
- Paso 9 — Extensión de REINFORCE con σ más pequeña (no pedida; reportada como extensión) 27
- Paso 10 — Ablación de las señales de observación (¿valen la pena todas?) 28
- Paso 11 — ¿Más entrenamiento u otro algoritmo cierra la brecha? 29
- Paso 12 — Fase 0 corregida: APSO con ε ~ U[0,1], PSO de inercia lineal y MSAPSO fiel sobre las curvas estáticas 30
- Paso 13 — REINFORCE con política Beta además de la gaussiana: diseño e implementación 31
- Paso 14 — Resultados: gaussiana vs Beta en REINFORCE 35

5. Conclusiones 39

6. Limitaciones, supuestos declarados y preguntas para el profesor 41

7. Mapa de archivos y cómo reproducir 43

Anexo. Código clave y especificación del MSAPSO implementado 45

RESUMEN

Resumen General y recomendaciones

Actualización V3 (18-oct): REINFORCE con política gaussiana vs Beta. A petición del profesor se repitió la rama REINFORCE (formas A y B; sin baseline, R−V, R−Q̂, Q̂−V) con dos distribuciones de política: la gaussiana de la V2 y una Beta, con la misma dispersión, la misma malla (480 configuraciones) y el mismo protocolo de selección en dos etapas (Pasos 13–14).

- **La conclusión de la V2 no cambia:** con ninguna de las dos distribuciones REINFORCE alcanza al MSAPSO fiel en transiciones de sombra. La mejor variante (Beta, forma B sin baseline) queda a −2.28 puntos de energía [−2.80, −1.66] en los 12 pares reales y a −5.64 [−6.95, −4.48] en 100 sintéticos.
- **La Beta mejora un poco la forma B del profesor** (comparación limpia, mismos hiperparámetros): +0.72 puntos [+0.31, +1.09] en estáticos (4/4) y +0.45 [+0.27, +0.65] en pares reales (11/12); en sintéticos no es demostrable. En los demás brazos no hay diferencia consistente.
- **La ganancia aparece donde el óptimo está cerca del límite de corriente** (Casos 1 y 3, b ≈ 0.95), donde la gaussiana tiene que recortar candidatos y la Beta no. La diferencia es pequeña porque la selección elige σ = 0.05–0.1, y con esa dispersión ambas distribuciones casi coinciden.
- **Siguiente paso:** aprender también la dispersión (σ o κ). Ahí la Beta, cuya varianza ya depende de μ y está acotada, es donde podría marcar diferencia.

Veredicto. El objetivo se cumple **parcialmente y en un grado bajo** (estimación del autor ≈ 25 %, no es una métrica). Hay una mejora pequeña y demostrable en convergencia estática, pero ninguna ante cambios de sombra, y una regla simple sin RL logra lo mismo.

Hallazgos

1. **La mejora de la V1 era un artefacto.** Medía mal la potencia (Vpv·Ipv, con valores > 100 %) y daba al agente información privilegiada; al corregirlo desapareció.
2. **Comparación justa construida:** MSAPSO fiel al paper (Fig. 6, Eq. 13 y 14), simulador numba ≈ 113× más rápido, observaciones sin información privilegiada, 3 semillas e IC bootstrap pareados.
3. **PPO contra el MSAPSO fiel (5 ms):** sombra estática **+1.84 pp** [+1.23, +2.42] (única mejora demostrable); 12 transiciones reales +0.42 [−1.35, +1.88]; 100 patrones sintéticos −0.82 [−2.04, +0.34] (estos dos, sin diferencia demostrable).
4. **A 0.5 ms** (valor literal del paper) no hay diferencia demostrable: la corriente del inductor no alcanza a seguir los escalones (L = 10 mH).

5. **Una regla sin RL** (β = 0.95 fijo, t_max = 5) iguala o supera a PPO en los casos reales (observación post-hoc, probada después de ver al agente).
6. **Otros métodos no mejoraron:** A2C y SAC iguales o peores; 3M pasos peor; el mejor REINFORCE (σ = 0.05, forma B, lr 0.001) obtuvo 95.6 / 93.9 / 89.9 % frente a 97.3 / 96.4 / 96.0 % del MSAPSO.
7. **Señales de observación:** las de caída de potencia sirven para detectar cambios de sombra, no para converger mejor en sombra estática.

Conclusión

Sí: mejora pequeña y demostrable en convergencia estática (+1.8 pp) y un marco de evaluación sólido y reutilizable. **No:** sin mejora demostrable ante cambios de sombra (donde el RL debía aportar), ni a 0.5 ms, y una regla simple logra lo mismo. Queda establecido que la V1 sobreestimaba el RL y que el MSAPSO fiel ya está cerca del techo físico (≈ 96–97 %), lo que deja poco margen.

Recomendaciones para seguir

1. **Cambiar lo que controla el agente:** que elija directamente la siguiente corriente a probar, con memoria (PPO recurrente), entrenado en miles de patrones. En la V2 solo controlaba β y el reinicio.
2. **Actor-crítico asimétrico:** el crítico usa información privilegiada solo en entrenamiento; reduce la variabilidad entre semillas.
3. **Detección de cambio aprendida (RL jerárquico):** sin regla de reinicio el desempeño cae a 56.6 % y el umbral fijo de 5 W es frágil con ruido.
4. **Optimización bayesiana como control barato** para saber si el RL realmente aporta algo.
5. **Escenarios más exigentes:** ruido de medición, parámetros del panel cambiantes y picos estrechos, donde el MSAPSO fijo debería sufrir.

Sugerencia: hacer 1 y 2 juntos con el mismo arnés de la V2, usar 4 como control, y acordar con el profesor el régimen (0.5 o 5 ms) y la métrica de éxito válidos.

01

1. Resumen ejecutivo

Esta corrida (V2) rehace las Fases 0 a 3 atendiendo las observaciones del profesor y mis propias correcciones a la V1. El principio rector fue **comparar contra el MSAPSO tal como lo define el paper**, con el mismo simulador, la misma métrica y varias semillas, y reportar el resultado sea cual sea.

> **Respuesta corta a la pregunta del proyecto («¿vale la pena RL-MSAPSO?»).** Bajo una comparación fiel y justa, el agente de RL (PPO) **no supera de forma robusta al MSAPSO del paper en las transiciones de sombra**; su única ventaja demostrable es converger más rápido en sombras estáticas (≈ +1.8 puntos de energía), y esa ventaja la reproduce una regla trivial sin RL (β alto constante y búsqueda corta). La «mejora enorme» de la V1 (82.85 % → 99.33 %) **no se sostiene**: venía de un baseline débil, de una observación con información filtrada y de una métrica que no medía energía entregada.

Hallazgos principales

1. **Simulador V2 validado.** Mismo modelo físico (panel + Boost + Super-Twisting), núcleo compilado ≈ 113× más rápido; diferencia ≤ 0.003 % en potencia y 0 % en energía respecto a la V1 (Paso 1). El controlador deslizante SÍ está simulado (ecs. 4–7 del paper).
2. **MSAPSO fiel a la Fig. 6** (7 partículas, β 0.1→0.95, t_max = 25, barrido alternante, Gbest inmediato, reinicio a 5 W). Con t_wait = 5 ms alcanza 100 % de potencia final en los 4 casos y en las 12 transiciones (Pasos 2–3). Con el 0.5 ms del paper, la 1.ª búsqueda dura 87.5 ms (cuadra con la Tabla 3) pero solo 8 de 12 transiciones llegan a ≥ 99 %: el convertidor simulado no sigue referencias tan rápidas (límite de pendiente del inductor, L = 10 mH). Re-sintonizar el Super-Twisting no lo resuelve (Figura D).
3. **PPO vs MSAPSO fiel** (3 semillas, 5 ms; energía entregada, diferencia pareada con IC95 %): casos estáticos **+1.84** [+1.23, +2.42] (4/4); transiciones reales +0.42 [−1.35, +1.88]; sintéticas −0.82 [−2.04, +0.34]. En el régimen literal de 0.5 ms: −1.04, −2.54 y −1.21 (todos con IC que incluye 0). Hay variabilidad entre semillas (≈ 1 punto).
4. **Control sin RL.** Un MSAPSO con calendario de β optimizado de forma honesta (selección optimización → validación) ≈ el del paper (+0.3). Reglas simples probadas *después* de ver al agente (post-hoc): «β = 0.95 fijo, t_max = 5» da 99.29 / 98.37 / 95.71 y «β 0.5→0.95, t_max = 8» da 98.98 / 98.03 / 97.35 (estáticos / transiciones reales / sintéticas): igualan o superan al PPO (99.10 / 96.84 / 95.21).
5. **Qué aprendió el agente** (Figura F): arranca con β ≈ 0.95, colapsa el enjambre sobre el mejor punto de la 1.ª pasada (dispersión ≈ 0 en 2–3 pasadas frente a ≈ 12 del MSAPSO) y reinicia justo después del cambio de sombra. La «β alta primero y menor cerca del MPP»

solo aparece de forma leve y no tiene efecto, porque con el enjambre colapsado β no mueve nada.

6. **Señales de entrada** (ablación, Paso 10): la dispersión aporta poco (−0.4 a −0.6 puntos, dentro del ruido); «tiempo desde el reinicio» importa en sombras sintéticas (−3.5); las señales de variación de potencia importan en las transiciones (−1.5) pero NO en la convergencia de casos estáticos: sirven para detectar el cambio de sombra, no para acelerar la convergencia.
7. **Más entrenamiento u otros algoritmos** (Paso 11): 3 M de pasos no mejora (empeora la mejor semilla); A2C y SAC rinden parecido o peor. Ninguno supera de forma robusta al MSAPSO fiel en transiciones.
8. **REINFORCE gaussiano** (Pasos 5 y 9): converge a la potencia correcta pero con σ fija gasta más energía explorando. Con σ = 0.3–0.8 (lo propuesto) entrega ≈ 84–89 %; con σ = 0.05 (extensión) sube a 95.6 / 93.9 / 89.9 % (estáticos / transiciones / sintéticas), todavía por debajo del MSAPSO (97.3 / 96.4 / 96.0).
9. **Fase 0 corregida** (Paso 12): con ε ~ U[0,1] como en el paper, APSO resulta peor y más variable que con mi ε ~ U[−1,1]; la conclusión cualitativa se mantiene (el MSAPSO determinista no baja de 99.94 %, los estocásticos llegan a ≈ 86–88 % en su peor semilla).

Qué NO se demostró: que RL no pueda aportar nada en general. Se demostró que, en este simulador, con este problema (4 casos reales + sombras sintéticas, medición sin ruido) y con PPO/A2C/SAC por defecto, no hay ventaja robusta sobre el algoritmo del paper. Las limitaciones y los supuestos declarados están en la Sección 6.

2. Respuesta punto por punto a las observaciones del profesor

OBSERVACIÓN DEL PROFESOR	RESPUESTA	DÓNDE
En APSO, ε se muestrea en [−1, 1] pero el paper dice [0, 1].	**Correcto.** Además yo escalé el ruido por Isc·0.05 y fijé α y β por mi cuenta; mis PSO/APSO de la V1 no eran fieles. Se re-corrió con ε ~ U[0,1]: APSO resulta peor y más variable que con ε ~ U[−1,1]; la conclusión cualitativa (el MSAPSO determinista no baja de 99.94 %, los estocásticos llegan a ≈ 86–88 % en su peor semilla) se mantiene.	Paso 12
Los aumentos sobre el baseline se marcan con «β fijo»; tendría que ser β acelerado como en APSO.	El rótulo estaba mal (era una rampa 0.1→0.9, Eq. 14), pero el problema real era más profundo: ese baseline no era el MSAPSO del paper (5 partículas, 1 ms, sin reinicio, orden fijo…). Eso explica que no se encontraran los porcentajes del paper y que RL «mejorara drásticamente». En la V2 el baseline es el MSAPSO fiel con β de la Eq. 14.	Sección 3; Pasos 2–3, 8
Mostrar una gráfica de cómo cambia β conforme las partículas se acercan al MPP.	Hecha para el MSAPSO (β, dispersión, Gbest y potencia por pasada) y para el agente PPO frente al MSAPSO. Hallazgo: el agente arranca con β ≈ 0.95 y colapsa el enjambre en 2–3 pasadas; el MSAPSO tarda ≈ 12.	Figuras A y F
El delta de potencia como entrada del MLP puede hacer que saque una β alta para convergencia acelerada y la reduzca al llegar al MPP.	Se probó. Quitar las señales de variación de potencia NO afecta la convergencia rápida en casos estáticos (98.78 vs 99.15 %); el β alto inicial se aprende sin ellas. Sí afectan las transiciones (−1.5 puntos): sirven para detectar el cambio de sombra. La reducción de β cerca del MPP es leve e irrelevante (el enjambre ya está colapsado).	Figura F; Paso 10
Correr de nuevo APSO con ε corregido y comparar RL-MSAPSO con MSAPSO (β acelerado, no fijo).	Hecho: APSO corregido (Fase 0) y comparación de RL contra el MSAPSO fiel con β de la Eq. 14, además de un MSAPSO con calendario optimizado sin RL y reglas simples.	Pasos 7, 8, 12
¿Realmente se hizo un simulador del controlador por modos deslizantes? ¿Es real la gráfica de la diapositiva 9?	**Sí.** `converter_V2.py` integra la ley u = u_eq + u_st (−λ√|s|·sign(s) − z, ż = Y·sign(s), ecs. 4–7) sobre el modelo promediado del Boost; las gráficas salen de esa simulación, no están dibujadas (Figuras B y C muestran la corriente real del inductor siguiendo la escalera). Salvedades: modelo promediado (sin PWM a 45 kHz), sign suavizado con tanh(s/10^{-3}), planta I(V) interpolada de los datos, ganancias del paper; y con ellas el convertidor no sigue referencias de 0.5 ms. Supongo que la diapositiva 9 era la figura de la Fase 1 de la V1; no la he visto.	Paso 1, 3; Figuras B, C, D; Anexo
¿La validación es escalonada: un conjunto de menor a mayor, el siguiente de mayor a menor, y así?	En la V1 **no** (siempre ascendente). En la V2 **sí** (Figura B), con la 1.ª pasada de menor a mayor, como indicó el profesor. Dato de la ablación: alternar no es lo que hace viable el tiempo corto; lo decisivo es que Gbest sea la corriente medida (consistente con Pbest).	Paso 2–3; Figura B

OBSERVACIÓN DEL PROFESOR	RESPUESTA	DÓNDE
¿Valen la pena todos los inputs? ¿Por qué «tiempo desde el último reinicio» y «dispersión»?	Los elegí por intuición (la dispersión resume cuánto se colapsó el enjambre; el tiempo desde el reinicio es un reloj de la búsqueda). Sin evidencia entonces; la ablación la da: la dispersión es prescindible; el reloj importa en sintéticas; las variaciones de potencia importan en transiciones.	Paso 10; Figura J
¿Qué tal con distintos métodos de RL?	PPO (3 semillas, 1 M y 3 M de pasos), A2C y SAC (1 semilla). Ninguno supera de forma robusta al MSAPSO fiel en transiciones.	Pasos 8, 11
Probar MSAPSO con la fórmula del gradiente de la gaussiana (REINFORCE) con distintos baselines y tasas de aprendizaje; que sea MSAPSO escalonado.	Implementado: muestras gaussianas en escalera alterna, forma del profesor (B) y estimador de libro (A), baselines R–V, R–$\hat{Q}$, $\hat{Q}$–V, tasas 0.1 / 0.01 / 0.001 / acelerada / 1/t, $\sigma \in \{0.3, 0.5, 0.8\}$ y extensión con σ pequeña. Queda sin implementar el brazo «β/tasa por PPO» en el muestreador gaussiano (el PPO ya controla β del esquema SAPSO, que es el caso análogo).	Pasos 5, 9; Figura E
Reproducir el MSAPSO fielmente, incluida la parte escalonada; si no, la comparación no es justa.	Hecho a partir de la Fig. 6 y de lo indicado por el profesor; la especificación y las decisiones no cubiertas por el paper están declaradas (Anexo y Sección 6). La fidelidad no la certifico yo: el profesor debe validar la especificación.	Paso 2; Anexo
«Valdría la pena seguir verificando si sí serviría» RL-MSAPSO.	Respuesta y recomendaciones en la Sección 5.	Sección 5
«Me gustaría que probaran los modelos de RL que les comentaba tanto con gaussiana como con PDF de Beta distribution» (semana del 12-oct).	**Hecho (V3).** REINFORCE con las dos políticas, misma dispersión en $\mu = 0.5$ ($\kappa = 0.25/\sigma^2 - 1$), misma malla y protocolo. La Beta mejora un poco la forma B (+0.72 en estáticos y +0.45 en pares reales, IC sin cero); en los demás brazos no hay diferencia consistente, y ninguna variante alcanza al MSAPSO fiel en transiciones. PPO con política Beta queda pendiente.	Pasos 13–14; Figuras K–N

3. Qué estaba mal en la V1 y cómo se corrigió

Esta sección lista, sin atenuantes, lo que estaba mal o era inexacto en la corrida y en el documento de corroboración anteriores (V1), y qué se hizo en la V2 para cada punto. Algunos los señaló el profesor; otros los detecté al revisar el código y los reporté antes de rehacer el trabajo.

PROBLEMA EN LA V1	POR QUÉ IMPORTABA	CORRECCIÓN EN LA V2
ε de APSO en [−1, 1] y ruido escalado por Isc·0.05 (el paper dice [0, 1])	Mi PSO/APSO eran versiones propias; la afirmación «reproduce fielmente el paper» era exagerada.	Se reescribió el MSAPSO desde la Fig. 6. APSO/PSO ya no se usan como referencia principal; el APSO con ε ∈ [0,1] queda pendiente como referencia secundaria (ver Sección 6).
«Baseline β fijo» mal rotulado	En realidad era una rampa lineal 0.1→0.9 (Eq. 14), no un β fijo; el rótulo indujo a una lectura equivocada.	Los rótulos de la V2 describen exactamente cada configuración.
El baseline no era el MSAPSO del paper: 5 partículas, β_max 0.9, 30 iteraciones, ~1 ms de espera, barrido siempre ascendente, sin criterio de reinicio, Gbest actualizado al final de cada conjunto	El paper reporta >99 % en las combinaciones; mi baseline sacaba 58–88 %. Un baseline más débil que el algoritmo real inflaba la mejora de RL (de 82.85 % a 99.33 %).	MSAPSO fiel a la Fig. 6: 7 partículas Isc·[0.95…0.1], β 0.1→0.95, t_max 25, barrido alternante, Gbest inmediato, reinicio si |Pbest−Ppv| ≥ 5 W. Se verificó en el simulador (Pasos 2–3).
Fuga de información en la observación de RL: señales normalizadas por el GMPP real y un indicador directo de «cambió la sombra»	Un controlador real no conoce el GMPP ni recibe un aviso de cambio de sombra; el agente podía reiniciar exactamente cuando se le avisaba. Mi comentario en el código decía lo contrario, y el documento de corroboración describía la observación de forma inexacta.	Observación solo con señales medibles (potencia en unidades de 100 W, caída relativa respecto a Pbest, dispersión, etc.). El GMPP/P_ceiling se usa únicamente en la recompensa de entrenamiento.
Métrica: «mejor potencia encontrada» (Gbest) en vez de energía entregada	No penalizaba el tiempo de búsqueda ni la energía perdida mientras se explora, y podía inflarse con lecturas transitorias.	Métrica principal = energía entregada por el panel / energía ideal (P_ceiling · tiempo). La recompensa de RL es la misma magnitud.
Se medía contra el GMPP teórico aun cuando era inalcanzable con el Boost y R = 12 Ω (Caso 3 y varias curvas sintéticas)	Castigaba a todos los métodos por algo físicamente imposible.	Toda eficiencia se mide contra P_ceiling (máximo alcanzable con V/I ≤ R); coincide con el GMPP cuando este es alcanzable.

PROBLEMA EN LA V1	POR QUÉ IMPORTABA	CORRECCIÓN EN LA V2
Gbest guardaba la corriente comandada (la referencia), no la medida	Con ventanas cortas el convertidor no ha llegado a la referencia: Pbest y Gbest dejaban de ser el mismo punto de la curva y aparecían reinicios espurios.	Gbest = corriente de salida del módulo medida (la «Ipv» del paper, Eq. 1) y Ppv = Vpv·Ipv en los terminales del panel.
Una sola semilla de entrenamiento y 20 episodios aleatorios	No permite distinguir mejora real de variabilidad.	3 semillas por régimen, desviación entre semillas y diferencia pareada por escenario con IC95 % bootstrap.
Simulador lento (solve_ivp) que hacía inviable usar los tiempos del paper	Condicionó la elección del tiempo de espera y de la longitud de las pruebas.	Mismo modelo físico con núcleo compilado (≈113× más rápido), validado contra la V1 (≤ 0.003 % de diferencia).

4. Paso a paso de las corridas V2 y V3

Cada paso indica **qué se hizo**, **por qué** (el razonamiento), **cómo se verificó**, el **resultado** y los **archivos** que lo respaldan. Es el registro cronológico de lo realizado; las figuras se insertan después del paso donde se generaron.

V3 (2026-10-18). Esta bitácora es la de la V2 sin cambios, más los Pasos 13 y 14 al final: REINFORCE con política Beta además de la gaussiana, a petición del profesor. La V3 solo agrega archivos con sufijo `_V3`; no modifica ningún archivo de la V2.

Documento vivo: cada entrada registra QUÉ se hizo, POR QUÉ (razonamiento), QUÉ RESULTÓ y QUÉ ARCHIVOS lo respaldan. De aquí se compila el PDF final para el profesor.

Contexto y punto de partida (2026-10-10)

- La V1 (carpeta `Proyecto Integrador Paneles/`) NO se modifica; se conserva como evidencia.
- Observaciones del profesor sobre la V1: (1) ε de APSO muestreado en [-1,1] en vez de [0,1]; (2) el "baseline" no era el MSAPSO del paper; (3) falta graficar β; (4) cuestionó si el controlador deslizante realmente se simuló; (5) pidió barrido escalonado alternante fiel al paper; (6) propuso rama REINFORCE (gradiente gaussiano) con baselines R-V(s), R-Q(a,s), Q(a,s)-V(s).
- Autocrítica propia previa (reportada al profesor): fuga de información en la observación de RL (normalización por P_gmpp y aviso de cambio de sombra), baseline con 5 partículas/1 ms/sin reinicio, métrica = mejor potencia encontrada (no energía entregada), una sola semilla.
- Lectura directa de las figuras 5 y 6 del paper (como imagen, no como texto): MSAPSO = 7 partículas Isc·[0.95 0.8 0.65 0.5 0.3 0.15 0.1], β_min=0.1, β_max=0.95, t_max=25, t_wait=0.5 ms, pasada "positiva" y pasada "negativa" alternadas, Gbest se actualiza al instante (Ppv ≥ Pbest), posiciones recalculadas al final de cada pasada (Eq.13), y reinicio si |Pbest−Ppv| ≥ 5 W medido tras t_max con el convertidor fijo en Gbest.

Decisiones declaradas para los puntos que el profesor no pudo aclarar (fin de semana)

DUDA	DECISIÓN	COBERTURA
Dirección de la 1.ª pasada	Menor→mayor corriente (dicho por el profesor), luego mayor→menor, alternando	Variante con orden literal de la Fig. 6 como sensibilidad
¿Eq.12 (ε) o Eq.13?	Eq.13 determinista (el paper dice que MSAPSO elimina la aleatoriedad)	APSO aparte con ε∈[0,1]
Reinicio	Solo tras t_max, umbral absoluto 5 W (Fig. 6)	Variante "detección temprana" (cada pasada) y "sin reinicio"

DUDA	DECISIÓN	COBERTURA
t_wait 0.5 ms vs 5 ms	Probar el barrido fiel a 0.5 ms; re-sintonizar el controlador solo si no asienta	Reportar ambos tiempos
Tiempo de espera del agente RL	Fijo e igual para todos en la comparación principal	Ablación aparte
REINFORCE	Forma del profesor (principal) y forma de libro (control); corrientes en por unidad (÷Isc)	Malla en etapas

PASO 1

Paso 1 — Simulador V2 (mismo modelo físico, núcleo compilado) y validación contra la V1

Qué hice. Reescribí el modelo dinámico PV+Boost+Super-Twisting (`pvsim_V2/converter_V2.py`) con integración RK4 de paso fijo compilada con numba (la V1 usaba `scipy.solve_ivp`). Añadí a las curvas (`pvcurve_V2.py` , `array_model_V2.py`) el atributo `P_ceiling` (potencia máxima alcanzable con la carga R=12 Ω) y guardé las tablas I(V) listas para el núcleo compilado.

Por qué. (a) Con los tiempos fieles al paper (7 partículas × 0.5 ms por pasada, 25 pasadas, miles de episodios) la V1 era inviable (~2.8 ms de cómputo por ms simulado). (b) El profesor cuestionó si el controlador deslizante se simuló de verdad: aquí la ley u = u_eq + u_st con integrador z sigue siendo la del paper (ecs. 4–7); solo cambió el integrador numérico. (c) `P_ceiling` : en la V1 medí contra el GMPP teórico aunque fuera físicamente inalcanzable con un Boost y R=12 Ω (Caso 3), lo que castigaba a cualquier método por algo imposible.

Cómo lo verifiqué (`validar_simulador_V2.py`): - Convergencia en dt: con dt = 1e-5 s el error en P_final es 0.0001 % respecto a dt = 1e-6 s. - Equivalencia con la V1 (escalera de 7 corrientes × 3, ventanas de 0.5 ms y 5 ms, 4 casos): diferencia máxima en potencia ≤ 0.003 %, en energía 0.000 %. - Velocidad: V1 = 2.81 ms de cómputo por ms simulado; V2 = 0.025 ms → **aceleración ≈ 113×** (1 s simulado ≈ 25 ms de cómputo).

Resultado. Simulador V2 validado; todo lo que sigue usa dt = 1e-5 s.

Archivos. `pvsim_V2/converter_V2.py` , `pvsim_V2/pvcurve_V2.py` , `pvsim_V2/array_model_V2.py` , `validar_simulador_V2.py` .

PASO 2

Paso 2 — MSAPSO fiel a la Fig. 6 y primera compuerta de validación

Qué hice. Implementé `pvsim_V2/msapso_V2.py`: 7 partículas Isc·[0.95 … 0.1], β de 0.1 a 0.95 (Eq. 14), t_max = 25, barrido alternante (la 1.ª pasada de menor a mayor, como indicó el profesor), Gbest actualizado al instante con la regla Ppv ≥ Pbest, posiciones recalculadas al final de cada pasada (Eq. 13 determinista), «park» en Gbest tras t_max y reinicio si |Pbest − Ppv| ≥ 5 W. Un «paso» de tiempo = una pasada = 7 ventanas de t_wait; el mantenimiento (park) dura lo mismo, para que la energía de todos los métodos se compare en la misma base.

Corrección de fidelidad detectada en el camino. Primero guardé como Gbest la corriente del *inductor*; en la Fig. 6 el paper guarda `Gbest = Ipv` y mide `Ppv = Vpv·Ipv`, y su Eq. 1 define Ipv como la corriente de salida del *módulo*. En mi modelo (que incluye el capacitor de entrada Ce) ambas difieren en los transitorios, y entonces Pbest y Gbest no eran el mismo punto de la curva → reinicios espurios. Corregido: Ipv = I_panel(Vpv) (sensor en los terminales del panel).

Resultado (casos estáticos, 1 s, MSAPSO fiel, t_wait = 0.5 ms). Potencia final 100 / 100 / 100 / 97.5 % del techo alcanzable (casos 1–4); energía entregada 99.6 / 99.7 / 99.7 / 97.2 %; la 1.ª búsqueda termina en 87.5 ms = 25 × 7 × 0.5 ms (cuadra con los 0.12 s de la Tabla 3 del paper). Con 5 ms: 100 % en los 4 casos.

Archivos. `pvsim_V2/msapso_V2.py`, `evaluar_baselines_V2.py`, `resultados_V2/baselines_V2.txt|json`.

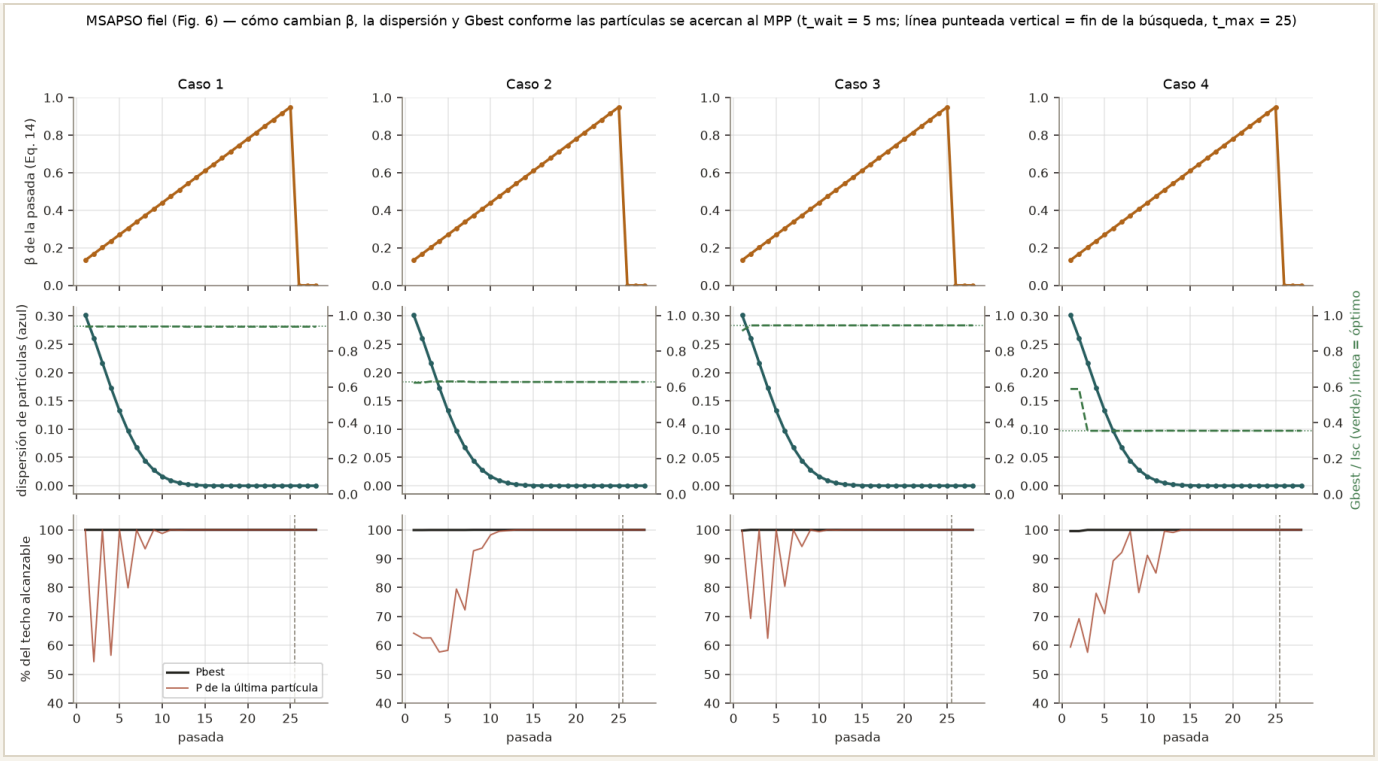


Figura A. MSAPSO fiel: β (Eq. 14), dispersión de las partículas, Gbest y potencia por pasada, en los 4 casos reales (t_wait = 5 ms). Responde a la petición de graficar β conforme las partículas se acercan al MPP: Gbest se fija en las 1–3 primeras pasadas y la dispersión llega a ~0 hacia la pasada 12; de la 13 a la 25 el enjambre está colapsado.

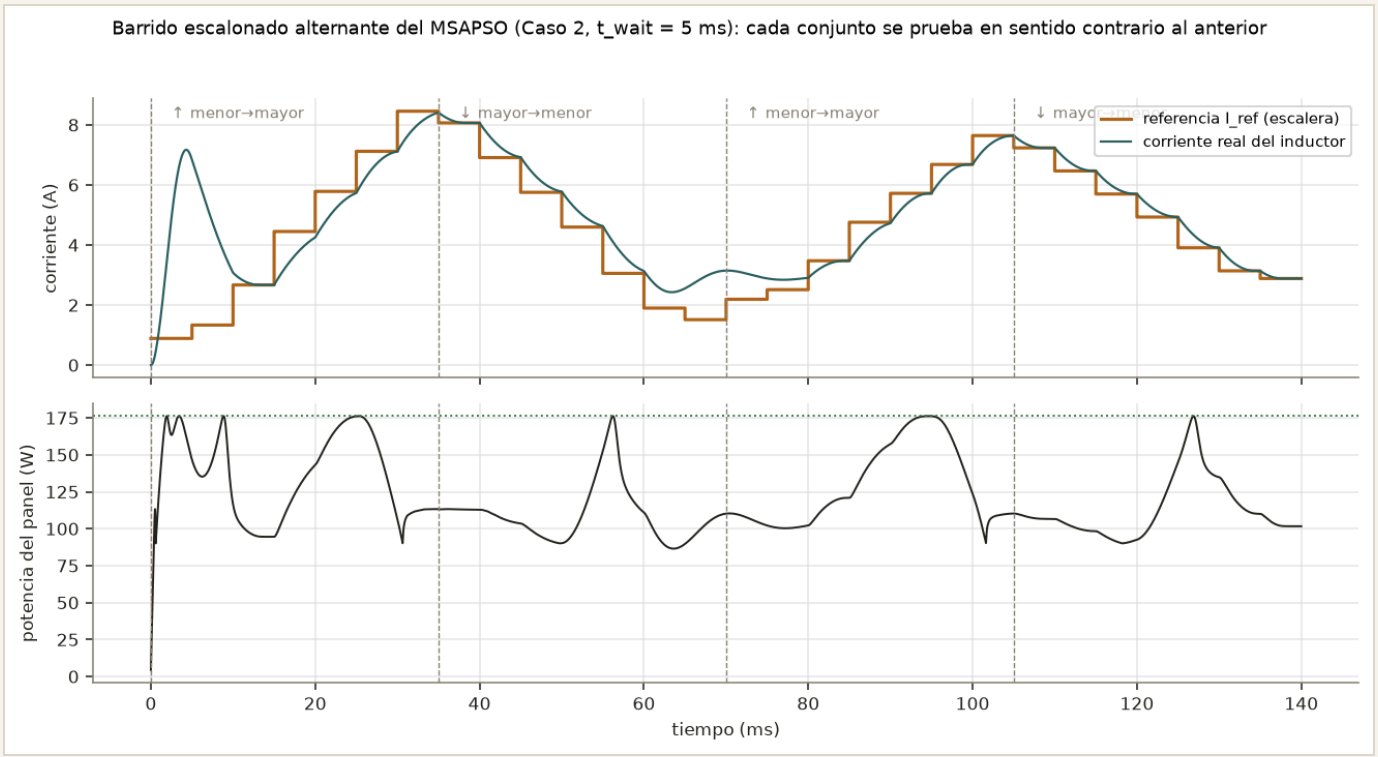


Figura B. Barrido escalonado alternante (Caso 2, t_wait = 5 ms): un conjunto se prueba de menor a mayor corriente, el siguiente de mayor a menor. Se muestra la referencia I_ref (escalera), la corriente real del inductor que entrega el Super-Twisting simulado y la potencia del panel.

PASO 3

Paso 3 — Ablación, diagnóstico de la dinámica del convertidor y decisión del régimen de tiempo

Hipótesis que probé y que resultó FALSA. Pensé que el barrido alternante era lo que hacía viable t_wait = 0.5 ms. Ablación a 0.5 ms (potencia final % / reinicios): alternar o no apenas cambia nada (100/100/100/97.2); lo decisivo fue guardar Gbest como la corriente *medida* (la V1 guardaba la corriente *comandada*): sin eso, 78.7 / 99.3 / 70.6 / 93.8 con reinicios espurios.

Diagnóstico del Caso «2→1» (transición que fallaba). Inspeccionando ventana por ventana: a 0.5 ms la corriente del inductor se mueve solo ~0.6 A hacia abajo y ~0.3 A hacia arriba por ventana; el convertidor no alcanza a seguir la referencia y la potencia medida refleja el punto donde el convertidor *va*, no el candidato aplicado. El algoritmo actúa como un «escáner de trayectoria».

¿Re-sintonizar el controlador ayuda? (`resintonia_controlador_V2.py`). Con las ganancias del paper (λ=0.1, Y=0.01): −0.59 A / +0.33 A por 0.5 ms. Con 10× las ganancias el convertidor ya satura en su límite físico (−1.00 A / +1.75 A; límite teórico −0.95 / +1.50 A con L = 10 mH, Vpv ≈ 30 V, Vo ≈ 49 V); con 100× y 1000× no mejora. Conclusión: re-sintonizar solo da 2–3× y no alcanza; el límite es el inductor.

Transiciones (12 pares ordenados, MSAPSO fiel) vs t_wait: pares con potencia final ≥ 99 %: 8/12 (0.5 ms), 8/12 (1 ms), 9/12 (2 ms), **12/12 (5 ms)**; eficiencia de energía 95.1 → 96.4 %.

Decisión declarada. Régimen PRINCIPAL: t_wait = 5 ms (el convertidor sigue la referencia; todas las mediciones son honestas). Régimen SECUNDARIO: t_wait = 0.5 ms (valor literal del paper). Ambos con las ganancias del paper. Pregunta para el profesor (no bloquea): ¿su convertidor simulado asienta en 0.5 ms? Con L = 10 mH el nuestro no puede por límite físico de pendiente de corriente.

Otros hallazgos del mismo barrido de variantes (`evaluar_baselines_V2.py`): - «Reinicio temprano» (comparar el máximo de cada pasada contra Pbest) NO sirve como baseline más fuerte: se dispara en casi cada pasada (95 reinicios/s) porque en una pasada con partículas dispersas el máximo puede quedar ≥5 W por debajo de Pbest legítimamente. Se descarta y se reporta así. - «Sin reinicio»: 53 % de energía en las transiciones frente a 95 % con la regla del paper → el reinicio es lo que sostiene el desempeño ante cambios de sombra. - El «baseline de la V1» reconstruido en el simulador V2 (5 partículas, β 0.1→0.9, sin reinicio, orden fijo, Gbest = referencia, 1 ms) entrega solo 55–85 % de la energía: confirma que era un baseline débil (la V1 lo evaluaba con «mejor potencia encontrada», métrica que no penaliza eso).

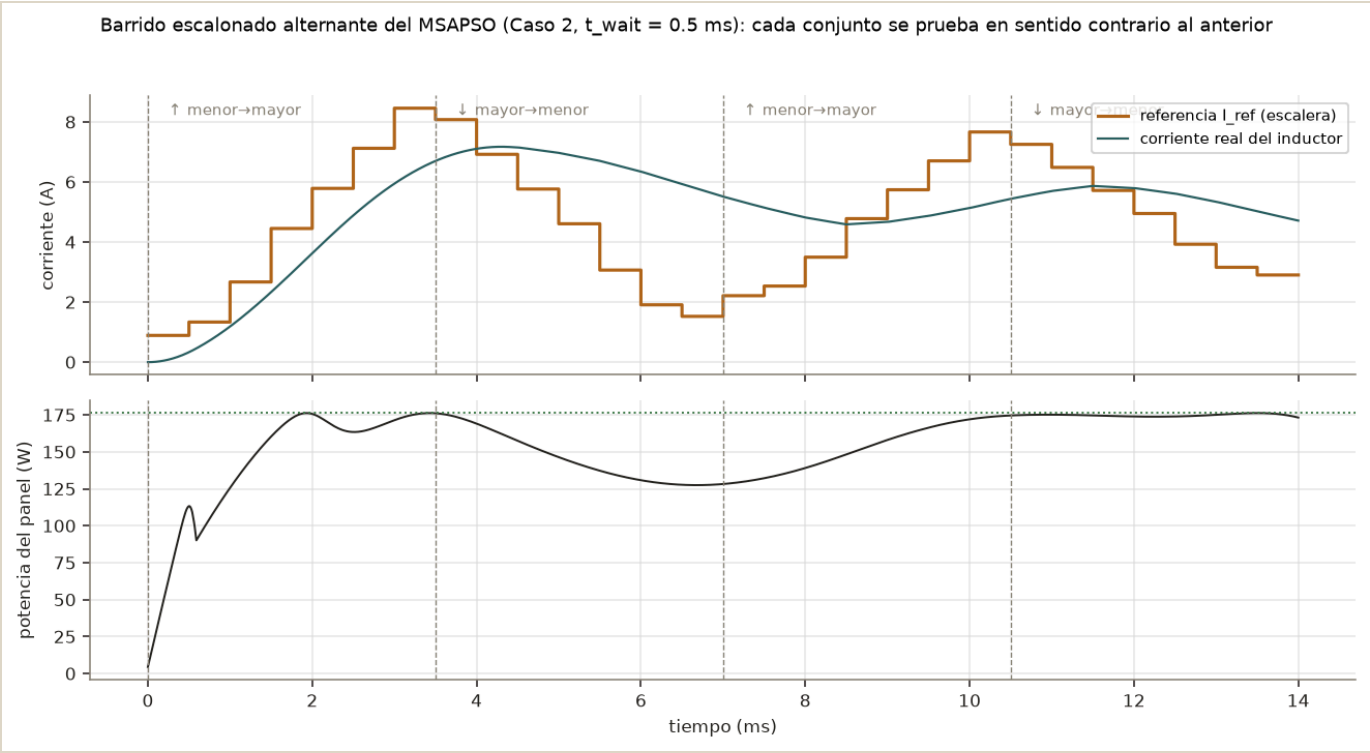


Figura C. Mismo barrido con t_wait = 0.5 ms: la corriente del inductor no alcanza a seguir los escalones (el convertidor está limitado por la pendiente de corriente con L = 10 mH).

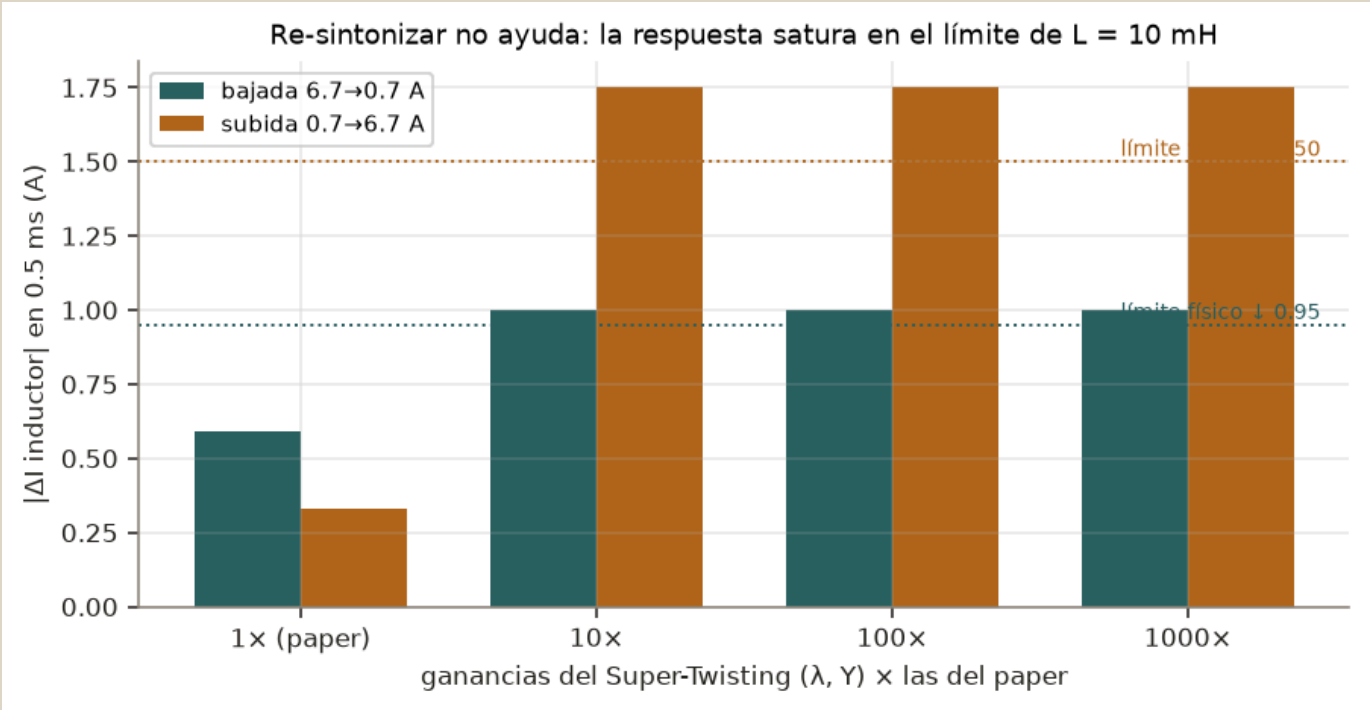


Figura D. Re-sintonizar el Super-Twisting: con ganancias ×10 el convertidor ya satura en su límite físico; ×100 y ×1000 no mejoran (Caso 1, escalón 6.7 ↔ 0.7 A).

PASO 4

Paso 4 — Entorno de RL V2, arnés común de evaluación y primera prueba de PPO

Qué hice. - `pvsim_V2/passrunner_V2.py` : la mecánica de UNA pasada (7 ventanas, barrido alternante, Gbest inmediato, Eq. 13) en una sola clase usada tanto por el entorno de RL como por los controladores que se evalúan → lo que aprende el agente y lo que se mide son idénticos. - `pvsim_V2/gym_env_V2.py` (Gymnasium; `check_env` OK): acción = [β ∈ 0.05–0.95, reinicio sí/no]; el tiempo de espera NO lo decide el agente (fijo, igual para todos los métodos); episodios de 130 pasadas con búsqueda inicial, un cambio abrupto de sombra (80 % de los episodios, entre las pasadas 20 y 70) y re-búsqueda. - **Observación sin información privilegiada** (8 señales: P_max y P_media de la última pasada, Pbest, caída relativa respecto a Pbest, dispersión de partículas, pasadas desde el reinicio, β previo, ΔPbest). Se eliminó la normalización por GMPP real y el aviso directo de cambio de sombra que tenía la V1. - **Recompensa = energía entregada en la pasada / energía ideal** (usa P_ceiling solo aquí, en entrenamiento). Premia lo que importa y castiga explorar de más; reemplaza la métrica «mejor potencia encontrada» de la V1. - Sombras de entrenamiento más exigentes (`random_shading_mix` : mezcla de sub-strings soleados y sombreados). Los 4 casos reales y sus pares NO se usan para entrenar (holdout). - `pvsim_V2/harness_V2.py` + `evaluar_rl_V2.py` : mismos escenarios y métrica para todos los métodos: 4 casos estáticos; 12 pares ordenados de casos reales (holdout); 100 pares de curvas sintéticas nuevas (semilla fija) para generalización. Segmentos de 62 pasadas.

Por qué. Responde a: «baseline injusto», «features que filtran información», «métrica que no mide la energía entregada» y «una sola semilla» (se entrenarán 3).

Velocidad. ≈1,000 pasos/s por entorno a t_wait = 5 ms y ≈8,000 a 0.5 ms → 1 M de pasos de PPO ≈ 8 min (la V1 tardaba ~3.3 h por 300 k). Por eso el entrenamiento se corre aquí, sin necesidad de VS Code.

Primer resultado (modelo de prueba, 1 semilla, 200 k pasos, t_wait = 5 ms; energía % / potencia final % / reinicios): | Método | 4 casos estáticos | 12 transiciones reales | 100 sintéticas nuevas | |---|---|---|---| | MSAPSO fiel (paper) | 97.26 / 100.0 / 0 | 96.41 / 100.0 / 1.00 | 96.04 / 99.23 / 0.95 | | MSAPSO sin reinicio | 97.26 / 100.0 / 0 | 56.58 / 56.54 / 0 | 60.76 / 60.66 / 0 | | PPO (prueba 200 k) | 99.35 / 99.93 / 0 | 96.54 / 98.00 / 0.67 | 93.14 / 94.35 / 0.65 |

Lectura. Con un baseline justo el panorama cambia por completo respecto a la V1: el agente gana en energía en casos estáticos (converge más rápido), EMPATA en las transiciones reales y queda POR DEBAJO en sombras sintéticas nuevas (modelo todavía subentrenado). La «mejora enorme» de la V1 venía del baseline débil y de la información filtrada, no del aprendizaje.

Siguiente. Entrenamientos formales (3 semillas × 2 regímenes de tiempo + ablación de señales), un control sin RL con el calendario de β optimizado, y la rama REINFORCE.

Archivos. `pvsim_V2/passrunner_V2.py`, `pvsim_V2/gym_env_V2.py`, `pvsim_V2/harness_V2.py`, `train_V2.py`, `evaluar_rl_V2.py`, `lanzar_entrenamientos_V2.sh`, `resultados_V2/eval_prueba.json`.

PASO 5

Paso 5 — Rama REINFORCE (propuesta del profesor): implementación y malla en etapas

Qué hice. `pvsim_V2/reinforce_V2.py` : cada pasada se muestrean K = 7 corrientes candidatas de N(μ, σ²) (en por unidad: corriente/Isc, así σ = 0.3/0.5/0.8 son fracciones de Isc), se prueban en ESCALERA (pasada ascendente, luego descendente, Gbest inmediato) y se actualiza μ con el gradiente del log de la gaussiana, (b − μ)/σ². «Park» y regla de reinicio de 5 W idénticos al MSAPSO fiel, de modo que solo cambia la regla de actualización. Formas: **B** (la del profesor: lr·W·(Gbest − μ)/σ², con W = recompensa asociada a Gbest; baseline none | V) y **A** (estimador REINFORCE de libro, suma sobre muestras; baselines none | R−V | R−Q̂(a) | Q̂(a)−V con Q̂ = regresión kernel gaussiana). Recompensa R = P/P_norm, con P_norm = 250 W (nominal) o la mejor potencia vista. Tasas: fija (0.1/0.01/0.001), acelerada (Eq. 14), 1/t.

Por qué estas decisiones. (1) Por unidad: con σ en amperios, 0.5 A significaría cosas muy distintas con Isc de 3.2 A que con 8.9 A. (2) Paso efectivo c = lr·W/σ² (estabilidad: converge si 0 < c < 2). (3) Se agregó la forma A como control, porque la forma B del profesor es en la práctica SAPSO con paso escalado por W/σ².

Malla en etapas (`reinforce_grid_V2.py`): etapa 1 = 180 configuraciones puntuadas con sombras sintéticas de entrenamiento (semilla 7; 30 pares × 2 repeticiones); etapa 2 = la mejor de cada (forma, baseline) y la mejor de cada tasa (forma B) evaluadas en los conjuntos de prueba con 5 repeticiones. **Resultado** (energía % / potencia final %), t_wait = 5 ms, MSAPSO fiel como referencia (97.26/100, 96.41/100, 96.04/99.23 en estáticos/pares reales/sintéticos): | Configuración elegida | 4 estáticos | 12 pares reales | 100 sintéticos | |---|---|---|---| | B, sin baseline (lr acelerada, σ 0.3, R nominal) | 87.65 / 96.9 | 85.58 / 95.6 | 83.88 / 95.4 | | B, baseline V (lr 0.001, σ 0.3) | 87.07 / 98.7 | 84.25 / 98.2 | 84.16 / 93.3 | | A, sin baseline (lr 0.1, σ 0.3) | 88.29 / 98.5 | 85.89 / 96.6 | 86.58 / 96.5 | | A, R−V (lr acelerada, σ 0.3, R nominal) | 87.61 / 100.0 | 87.65 / 99.6 | 88.71 / 98.0 | | A, R−Q̂ (acelerada, σ 0.3) | 85.78 / 95.2 | 85.72 / 98.3 | 87.66 / 97.4 | | A, Q̂−V (1/t, σ 0.3) | 87.64 / 95.0 | 87.75 / 98.2 | 86.56 / 96.4 |

Lectura. Todas las variantes convergen (potencia final 93–100 %) pero ENTREGAN MENOS ENERGÍA (84–89 %) que el MSAPSO (96–97 %): con σ fija, los candidatos siguen dispersos alrededor de μ durante toda la búsqueda, mientras que MSAPSO contrae el enjambre y por eso desperdicia menos. En los seis casos la mejor σ fue la MENOR de las propuestas (0.3), señal de que un σ más pequeño o decreciente podría ayudar (extensión no pedida por el profesor, se prueba aparte y se reporta como tal). Entre baselines no hay diferencias grandes y consistentes (A-R−V es la mejor en pares reales y sintéticos); la tasa acelerada con σ = 0.3 tiene c = lr/σ² hasta ≈ 10.5 > 2 (inestable por construcción) y aun así es elegida: el recorte de μ a [0.02, 1] limita el daño.

Archivos. `pvsim_V2/reinforce_V2.py`, `reinforce_grid_V2.py`, `resultados_V2/reinforce_grid_V2.json`.

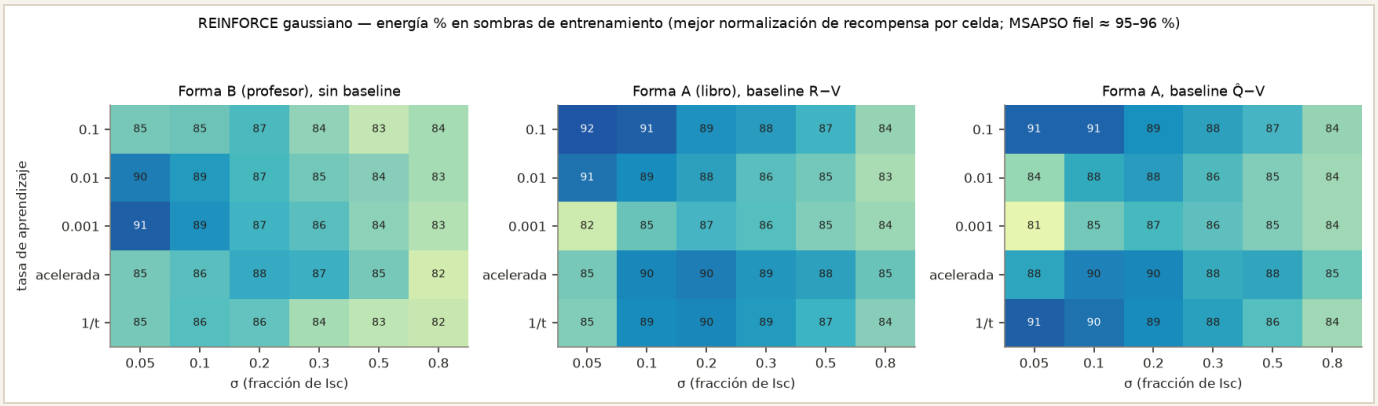


Figura E. REINFORCE gaussiano: energía % en sombras de entrenamiento según tasa de aprendizaje y σ (mejor normalización de recompensa por celda). La región buena es σ pequeña con paso efectivo c = lr/σ² moderado (0.3–1).

PASO 6

Paso 6 — Primer PPO formal (semilla 0) y qué aprendió el agente

Qué hice. Entrené PPO (stable-baselines3, hiperparámetros por defecto, red 64×64) 1,000,000 pasos, 4 entornos paralelos, t_wait = 5 ms (950 s de cómputo con la máquina cargada). Evaluación con el arnés común (`evaluar_rl_V2.py` ; energía % / potencia final %): estáticos 99.32 / 99.90 · pares reales 98.26 / 99.94 · sintéticos 96.57 / 98.53, frente al MSAPSO fiel 97.26 / 100 · 96.41 / 100 · 96.04 / 99.23. → ventaja modesta pero consistente en energía (+2.1, +1.9, +0.5 puntos), con potencia final equivalente.

Gráfica pedida por el profesor (β vs convergencia, `fig_agente_vs_msapso_s0.png`). Lo que hace el agente: arranca con β ≈ 0.95 → el enjambre se colapsa sobre el mejor punto de la PRIMERA pasada (la dispersión cae a ~0 en 2–3 pasadas; el MSAPSO tarda ~12) y reinicia justo después del cambio de sombra. La «β alta primero y menor cerca del MPP» que supuso el profesor aparece solo de forma leve (un valle ≈ 0.45 hacia la pasada 4) y no tiene efecto: con el enjambre ya colapsado, β no mueve nada.

Control sin RL (hallazgo clave, marcado POST-HOC porque lo probé después de ver al agente). MSAPSO con β = 0.95 constante y t_max = 3–5 entrega 99.29 / 99.86 (estáticos), 98.37 / 99.91 (pares reales) y 95.7 / 97.1 (sintéticos): IGUALA al PPO en casos reales; y «β 0.5→0.95, t_max = 8» da 98.98 · 98.03 · 97.35, superando al PPO en sintéticos. Es decir, la ventaja del agente sobre el calendario del paper se explica por un calendario más agresivo, no por aprendizaje.

Autocrítica sobre un intento previo de control. La primera búsqueda de un «MSAPSO optimizado» (121 configuraciones, 40 escenarios) eligió (β 0.28→0.60, t_max = 5), que mejoró en las sombras de optimización (96.22 % vs 95.04 %) pero EMPEORÓ en prueba (93.05 % vs 97.26 % en estáticos): sobreajuste por seleccionar el mejor de muchos sobre pocos escenarios (maldición del ganador). Se repite con espacio ampliado y selección en dos etapas (optimización → validación con 150 pares nuevos).

Archivos. `modelos_V2/ppo_5ms_full_s0.*` , `resultados_V2/eval_main_s0.json` , `resultados_V2/fig_agente_vs_msapso_s0.png` , `optimizar_msapso_V2.py` .

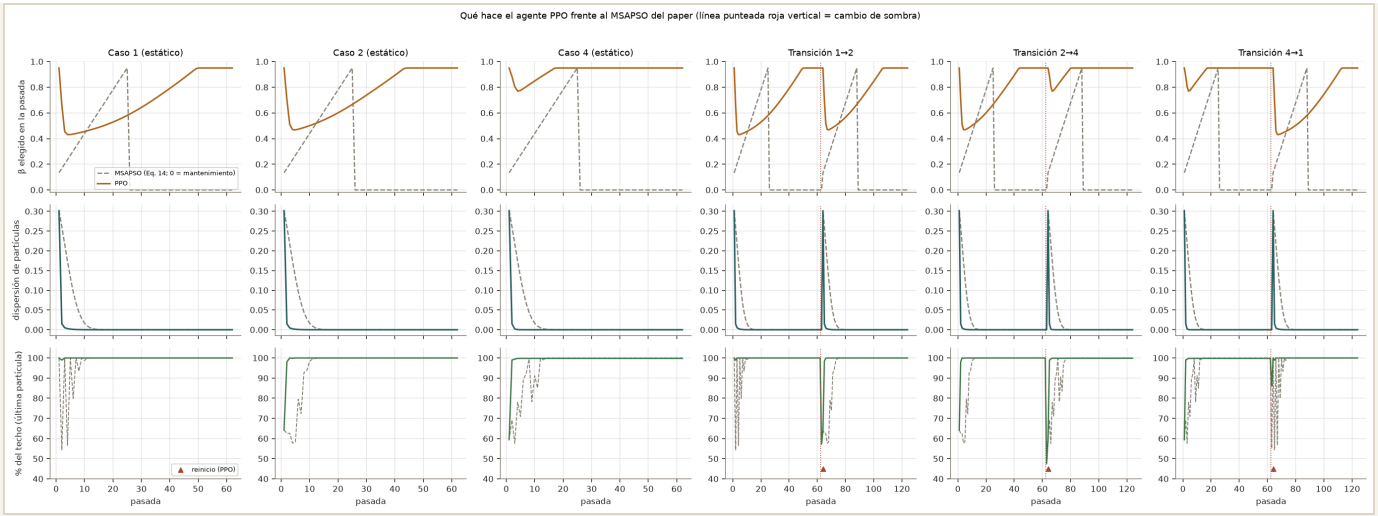


Figura F. Qué hace el agente PPO (semilla 0) frente al MSAPSO del paper: β elegido por pasada, dispersión de partículas, reinicios y potencia, en tres casos estáticos y tres transiciones.

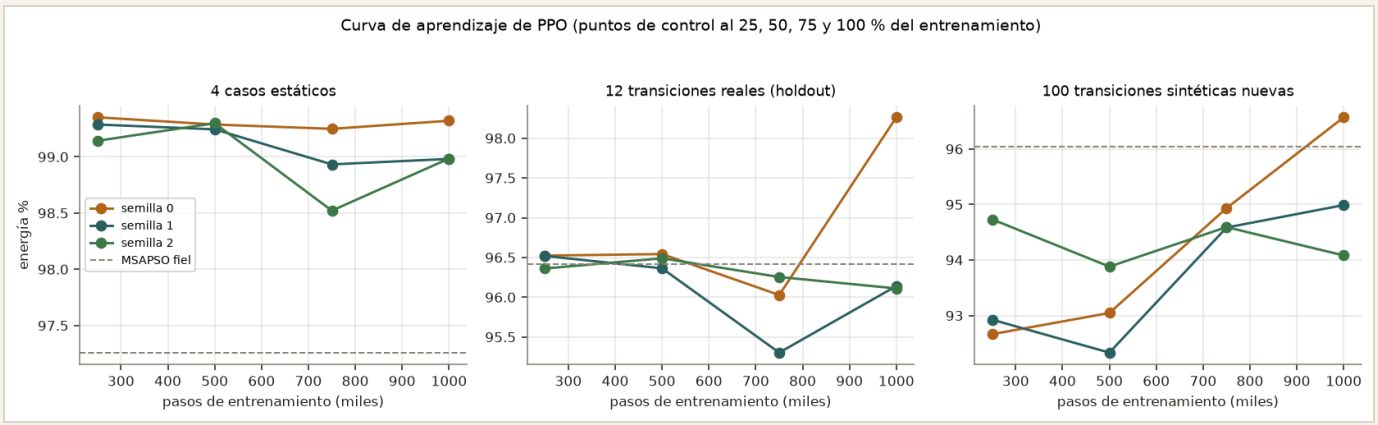


Figura G. Curva de aprendizaje de PPO por semilla (energía % en los tres conjuntos de prueba vs pasos de entrenamiento) y referencia del MSAPSO fiel.

PASO 7

Paso 7 — Control sin RL con selección en dos etapas (sin mirar los conjuntos de prueba)

Qué hice. `optimizar_msapso_V2.py` : búsqueda aleatoria de (β_min, β_max, t_max) con 150 configuraciones, espacio ampliado (β_min hasta 0.95; t_max ∈ {3, 5, 8, 12, 18, 25}), puntuadas con 60 pares sintéticos de OPTIMIZACIÓN (semilla 7); las 8 mejores + la del paper se re-puntúan con 150 pares de VALIDACIÓN (semilla 8) y se elige por validación. La evaluación en prueba (casos reales y sintéticas nuevas, semilla 2026) se hace una sola vez al final.

Por qué. Es el control honesto de «¿se explica la ganancia de RL por un mejor calendario de β?», evitando la maldición del ganador del primer intento (Paso 6).

Resultado. Mejor configuración (β 0.185 → 0.732, t_max = 25): validación 97.04 % vs 96.78 % del paper; prueba (energía %): 97.58 · 96.64 · 96.31 (estáticos · pares reales · sintéticos) vs 97.26 · 96.41 · 96.04 del paper → +0.3 puntos, es decir, prácticamente la configuración del paper. **Matiz importante:** sobre sombras sintéticas un calendario agresivo no gana, pero sobre los casos reales del paper sí (reglas simples post-hoc del Paso 6). Mi primera explicación («los 7 puntos iniciales de la rejilla caen cerca del GMPP en los casos reales») resultó FALSA al medirla: el mejor punto de la rejilla inicial alcanza solo 99.7 / 85.5 / 99.5 / 91.0 % del techo alcanzable en los casos 1–4 (y 96.0 % en promedio en las curvas sintéticas, con 40 % de ellas por debajo de 95 %). En el simulador dinámico las mediciones al final de cada ventana caen sobre la trayectoria del convertidor, no solo en las corrientes comandadas, así que una sola pasada explora más de 7 puntos; es plausible que eso ayude, pero no lo investigué a fondo. Por qué un calendario agresivo favorece a los casos reales y no a los sintéticos queda como PREGUNTA ABIERTA.

Archivos. `optimizar_msapso_V2.py` , `resultados_V2/msapso_optimizado_5ms.json` .

PASO 8

Paso 8 — PPO con 3 semillas y dos regímenes de tiempo (resultado principal)

Qué hice. Entrené PPO (hiperparámetros por defecto, red 64×64, 1 M de pasos, 4 entornos) con 3 semillas en t_wait = 5 ms (régimen principal) y 3 semillas en t_wait = 0.5 ms (literal del paper), y evalué todo con el arnés común (`analisis_final_V2.py` : media ± desv. entre semillas y diferencia pareada por escenario con IC95 % bootstrap).

Resultado, t_wait = 5 ms (energía %; potencia final % entre paréntesis): | Método | 4 estáticos | 12 pares reales | 100 sintéticos | |---|---|---|---| | MSAPSO fiel (paper) | 97.26 (100.0) | 96.41 (100.0) | 96.04 (99.2) | | MSAPSO sin reinicio | 97.26 (100.0) | 56.58 (56.5) | 60.76 (60.7) | | MSAPSO optimizado formal | 97.58 (100.0) | 96.64 (100.0) | 96.31 (99.2) | | regla simple β = 0.95, t_max = 5 (post-hoc) | 99.29 (99.9) | 98.37 (99.9) | 95.71 (97.0) | | regla simple β 0.5→0.95, t_max = 8 (post-hoc) | 98.98 (100.0) | 98.03 (100.0) | 97.35 (99.0) | | **PPO (3 semillas, media ± sd)** | **99.10 ± 0.16** (99.8) | **96.84 ± 1.01** (98.2) | **95.21 ± 1.03** (97.4) | Diferencia pareada PPO − MSAPSO fiel (puntos, IC95 %): estáticos +1.84 [+1.23, +2.42] (mejor en 4/4); pares reales +0.42 [−1.35, +1.88] (9/12); sintéticos −0.82 [−2.04, +0.34] (66/100). PPO − regla simple β = 0.95: estáticos −0.19, pares reales −1.53 [−3.29, −0.27] (la regla simple es significativamente mejor), sintéticos −0.50.

Hay variabilidad entre semillas: la semilla 0 llega a 98.26 % en pares reales; las semillas 1 y 2, a 96.1 %.

Resultado, t_wait = 0.5 ms (literal del paper): MSAPSO fiel 97.97 · 92.01 · 86.24; PPO 96.93 ± 0.78 · 89.48 ± 2.53 · 85.03 ± 0.33 (todas las diferencias con IC que incluye 0); las reglas simples agresivas FALLAN (92 · 71 · 76 y 97 · 68 · 80) porque el convertidor no sigue referencias que cambian tan rápido (límite de pendiente del inductor, Paso 3).

Lectura. Con una comparación justa, el agente de RL NO supera de forma robusta al MSAPSO del paper en las transiciones de sombra; su única ventaja demostrable es converger más rápido en sombras estáticas, y eso lo reproduce una regla trivial.

Archivos. `modelos_V2/ppo_*` , `analisis_final_V2.py` , `resultados_V2/final_raw_V2.json` , `resultados_V2/analisis_final_V2.txt` .

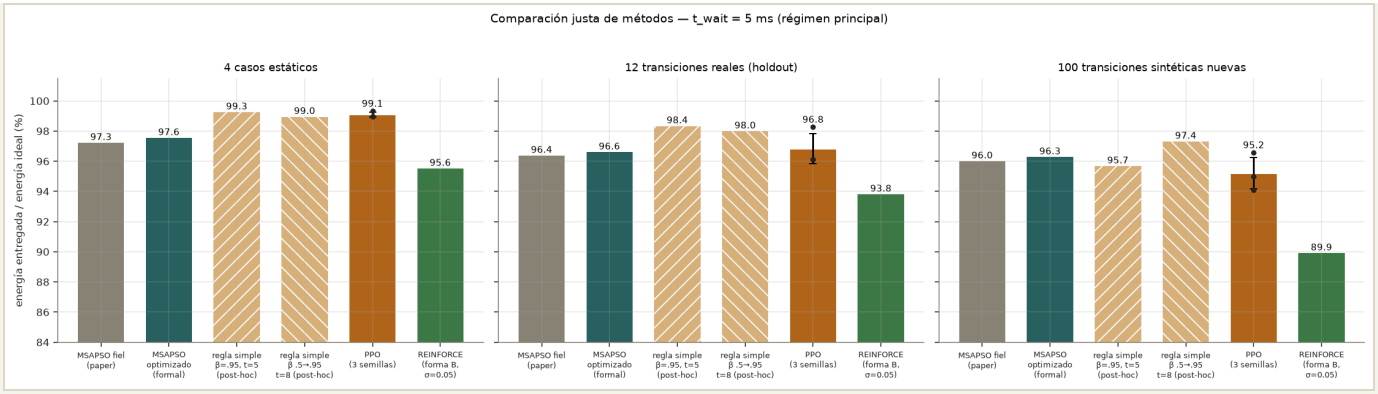


Figura H. Comparación de métodos, t_wait = 5 ms (régimen principal). Barras de error y puntos = desviación y valores por semilla de PPO. Las reglas «post-hoc» se probaron después de ver al agente.

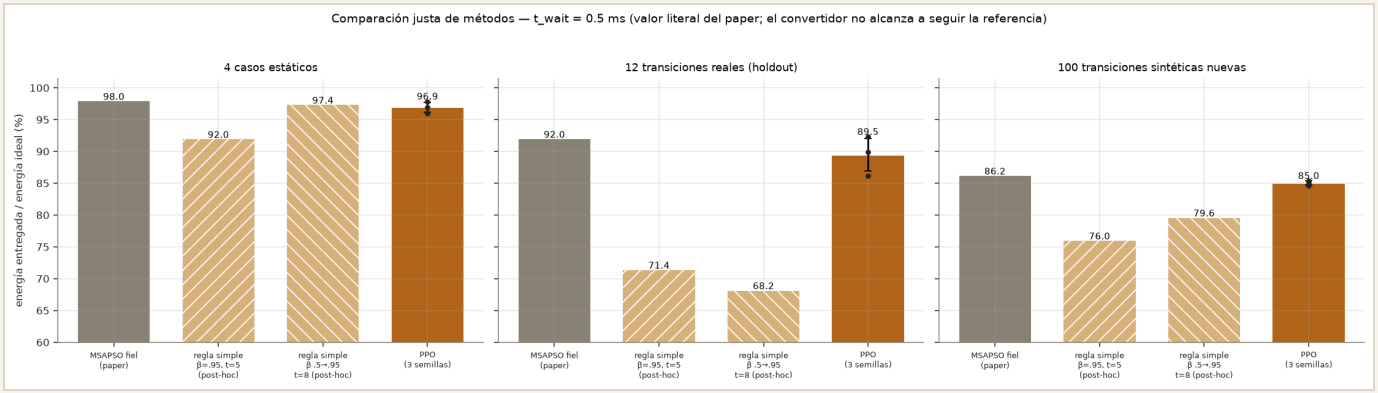


Figura I. Comparación en el régimen literal del paper (t_wait = 0.5 ms).

PASO 9

Paso 9 — Extensión de REINFORCE con σ más pequeña (no pedida; reportada como extensión)

Por qué. En la malla del Paso 5 la mejor σ siempre fue la menor propuesta (0.3). Probé σ ∈ {0.05, 0.1, 0.2} en cuatro variantes (B-ninguno, A-ninguno, A-R−V, A-Q̂−V) con 120 configuraciones.

Resultado (energía % / potencia final %; estáticos · pares reales · sintéticos): B-ninguno con lr = 0.001, σ = 0.05, R normalizada por la mejor potencia: **95.55/100 · 93.85/100 · 89.94/95.6** (paso efectivo c = lr/σ² = 0.4); A-R−V (lr 0.1, σ 0.05): 92.35 · 92.74 · 89.68; A-Q̂−V (1/t, σ 0.05): 91.90 · 93.05 · 89.84. Mejora grande respecto a σ ≥ 0.3 (≈ 88 %), pero sigue por DEBAJO del MSAPSO fiel (97.26 · 96.41 · 96.04), sobre todo en sintéticas. Conclusión: con σ fija, REINFORCE gaussiano converge a la potencia correcta pero gasta más energía explorando; un σ decreciente (no probado) sería el siguiente paso natural.

Archivos. `resultados_V2/reinforce_grid_ext_V2.json`.

PASO 10

Paso 10 — Ablación de las señales de observación (¿valen la pena todas?)

Qué hice. Entrené PPO (1 M de pasos, 2 semillas por variante, 5 ms) con subconjuntos de las 8 señales: completa; sin dispersión de partículas; sin «pasadas desde el reinicio»; sin las dos señales de variación de potencia (caída relativa respecto a Pbest y ΔPbest); y una mínima con solo P_max, Pbest y caída relativa. El profesor pidió justificar «tiempo desde el último reinicio» y «dispersión de las partículas», y sospechó que el delta de potencia explica el β alto inicial.

Por qué elegí originalmente esas señales (respuesta directa). La dispersión resume cuánto se ha colapsado el enjambre (historial acumulado de β) y sirve para decidir si conviene re-explorar; el tiempo desde el reinicio funciona como reloj de la búsqueda; las señales de variación de potencia son la pista para detectar un cambio de sombra. Fue intuición de diseño, sin evidencia previa; la ablación es la evidencia.

Resultado (energía % media de 2 semillas; estáticos · pares reales · sintéticos): completa 99.15 · 97.20 · 95.78; sin dispersión 98.73 · 96.68 · 95.19; sin t. desde reinicio 99.29 · 96.49 · **92.29**; sin caída/ΔPbest 98.78 · **95.74** · 95.10; mínima 99.24 · **94.89** · **92.55**.

Lectura (con cautela: la desviación entre semillas de PPO es ≈ 1 punto en pares y sintéticas). (1) En casos estáticos ninguna señal importa (≈ 99 %). (2) La **dispersión** aporta poco (−0.4 a −0.6 puntos, dentro del ruido): prescindible. (3) El **tiempo desde el reinicio** importa en sombras sintéticas (−3.5 puntos) y no en las reales. (4) Las **señales de variación de potencia** (caída relativa y ΔPbest) importan en las transiciones (−1.5 puntos en pares reales; −2.3 en la mínima): son la señal de detección del cambio. (5) Contra la hipótesis del profesor: quitar el delta NO afecta la convergencia rápida de los casos estáticos (98.78 vs 99.15); el β alto inicial se aprende sin él. El delta sirve para detectar el cambio, no para acelerar la convergencia.

Archivos. `modelos_V2/ppo_5ms_{no_disp,no_time,no_drop,minimal}_s{0,1}.*`, `resultados_V2/fig_ablacion_senales.png`.

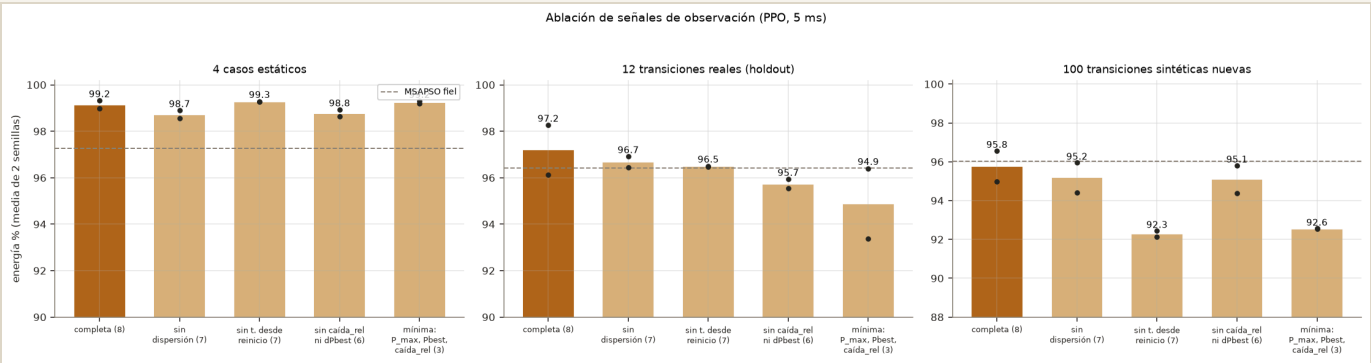


Figura J. Ablación de señales de observación (PPO, 5 ms; media de 2 semillas; puntos = semillas).

PASO 11

Paso 11 — ¿Más entrenamiento u otro algoritmo cierra la brecha?

Qué hice. (a) PPO con 3 M de pasos (2 semillas); (b) A2C 1 M de pasos y (c) SAC 300 k pasos (1 semilla cada uno), con hiperparámetros por defecto de stable-baselines3 y el mismo entorno y arnés. Respuesta a «¿qué tal les ha ido con distintos métodos de RL?».

Resultado (5 ms; energía % / potencia final %; estáticos · pares reales · sintéticos): | Método | 4 estáticos | 12 pares reales | 100 sintéticos | |---|---|---|---| | MSAPSO fiel (referencia) | 97.26 / 100.0 | 96.41 / 100.0 | 96.04 / 99.2 | | PPO 1 M (media de 3 semillas) | 99.10 / 99.8 | 96.84 / 98.2 | 95.21 / 97.4 | | PPO 3 M, semilla 0 | 98.82 / 98.3 | 95.56 / 98.0 | 94.11 / 98.5 | | PPO 3 M, semilla 1 | 98.91 / 100.0 | 96.79 / 100.0 | 93.68 / 96.8 | | A2C 1 M | 99.00 / 98.3 | 93.43 / 97.6 | 95.98 / 98.3 | | SAC 300 k | 98.80 / 99.9 | 96.75 / 99.9 | 95.03 / 97.5 |

Lectura. Triplicar el entrenamiento NO mejora (incluso la semilla 0, que con 1 M era la mejor, empeora de 98.26 a 95.56 en pares reales y hace reinicios innecesarios); A2C es peor en transiciones; SAC rinde parecido a PPO. Ningún algoritmo supera de forma robusta al MSAPSO fiel en las transiciones, y todos coinciden en converger más rápido en sombras estáticas. Es una sola semilla para A2C y SAC: son indicios, no conclusiones.

Archivos. `modelos_V2/ppo_5ms_full3M_s*`, `a2c_5ms_full_s0.*`, `sac_5ms_full_s0.*`, `resultados_V2/final_raw_V2.json`.

PASO 12

Paso 12 — Fase 0 corregida: APSO con ε ~ U[0,1], PSO de inercia lineal y MSAPSO fiel sobre las curvas estáticas

---

Qué hice. `fase0_msapso_V2.py` : sobre las curvas P(I) estáticas de los 4 casos (sin dinámica de convertidor, como la Fase 0 de la V1), corrí (a) el MSAPSO fiel (determinista); (b) APSO (Eq. 12) con **ε ~ U[0,1] como en el paper**, sin el factor Isc·0.05 que yo había añadido en la V1; (c) APSO con ε ~ U[−1,1] (la variante de la V1) para cuantificar el efecto de la discrepancia que señaló el profesor; (d) PSO de inercia lineal (ref. [40]: 10 partículas, t_max = 50, w 0.9→0.4, α = β = 2). Variable en por unidad (I/Isc); 30 semillas por caso en los estocásticos. α y β de APSO no vienen en el paper: α ∈ {0.05, 0.1, 0.2} (barrido declarado) y β = 0.4.

Por qué. Era la petición explícita del profesor («correr de nuevo APSO corrigiendo el muestreo de ε»). Se mide contra el GMPP teórico porque aquí no hay convertidor.

Resultado (eficiencia % sobre el GMPP; media / mínimo de 30 semillas; casos 1 · 2 · 3 · 4): | Algoritmo | Caso 1 | Caso 2 | Caso 3 | Caso 4 | |---|---|---|---|---| | MSAPSO fiel (determinista) | 100.00 | 100.00 | 100.00 | 99.94 | | APSO ε~U[0,1] (paper), α = 0.05 | 100.00 / 99.98 | 99.93 / 99.66 | 97.79 / 88.01 | 99.85 / 99.55 | | APSO ε~U[−1,1] (V1), α = 0.05 | 99.96 / 98.71 | 100.00 / 99.99 | 99.20 / 88.11 | 99.96 / 99.94 | | APSO ε~U[0,1] (paper), α = 0.2 | 99.92 / 99.75 | 99.48 / 97.75 | 94.74 / 87.92 | 98.87 / 86.01 | | APSO ε~U[−1,1] (V1), α = 0.2 | 100.00 / 99.98 | 99.99 / 99.93 | 96.82 / 88.04 | 99.94 / 99.74 | | PSO inercia lineal | 100.00 / 100.00 | 100.00 / 100.00 | 99.21 / 88.12 | 99.98 / 99.94 |

Lectura. (1) La corrección del profesor importa: con ε ∈ [0,1] el ruido es de un solo signo y empuja a las partículas siempre hacia corrientes mayores, por lo que APSO resulta peor y más variable que con mi ε ∈ [−1,1] (p. ej. Caso 3, α = 0.2: 94.7 % vs 96.8 %; Caso 4, mínimo 86.0 % vs 99.7 %). Mi APSO de la V1 era, pues, más favorable a APSO de lo que implica el paper. (2) La conclusión cualitativa de la V1 se mantiene: los algoritmos estocásticos (APSO, PSO) pueden caer a ~86–88 % en su peor semilla (quedan en el pico local del Caso 3), mientras que el MSAPSO fiel, por su siembra determinista en rejilla, no baja de 99.94 %. (3) Esta fase es sobre la curva estática: no dice nada del desempeño con dinámica real (Pasos 2–8).

Archivos. `fase0_msapso_V2.py` , `resultados_V2/fase0_V2.txt|json` .

PASO 13

Paso 13 — REINFORCE con política Beta además de la gaussiana: diseño e implementación

Petición del profesor (semana del 12-oct). «Me gustaría que probaran los modelos de RL que les comentaba tanto con gaussiana como con PDF de Beta distribution». Se interpreta como la rama REINFORCE que él propuso (Paso 5: formas A y B; sin baseline, R−V, R−Q̂, Q̂−V), corrida con dos familias de política: la gaussiana de la V2 y una Beta. PPO no se incluye en esta comparación: el profesor se refiere a «los modelos que les comentaba» (REINFORCE con baselines) y stable-baselines3 no trae una política Beta de serie; si se quiere, es el siguiente paso.

Por qué una Beta. La variable que se muestrea es la corriente candidata en por unidad, b = I/Isc, que físicamente vive en [0, 1]. La gaussiana tiene soporte en toda la recta y hay que recortar sus muestras a [0.02, 1]: cuando μ está cerca de un borde, una fracción grande de los candidatos cae exactamente en el límite (con σ = 0.2 y μ = 0.9, el 31 %; Figura K), se prueban corrientes repetidas y el gradiente se calcula sobre muestras recortadas, lo que lo sesga. La Beta tiene soporte (0, 1), no necesita recorte y su dispersión se encoge sola cerca de los bordes, lo que importa porque el óptimo alcanzable de dos de los cuatro casos reales está muy cerca del borde superior (b = I/Isc ≈ 0.94 en el Caso 1 y 0.95 en el Caso 3; 0.63 en el Caso 2 y 0.36 en el Caso 4).

Qué hice. `pvsim_V2/reinforce_dist_V3.py` hereda del controlador REINFORCE de la V2 y cambia solo dos cosas: cómo se muestrean los 7 candidatos y la función de puntaje (score) del gradiente de política. Todo lo demás (escalera alternante, Gbest inmediato, park, reinicio de 5 W, formas, baselines, tasas, normalización de la recompensa) es el mismo código.

- **Gaussiana:** b ~ N(μ, σ²) recortada a [0.02, 1]; score = (b − μ)/σ². Se conserva exactamente el cálculo de la V2 (score sobre la muestra recortada). Verificación: con la misma semilla reproduce la V2 bit a bit (9 corridas de prueba idénticas a 4 decimales).
- **Beta:** parametrización media-concentración, b ~ Beta(μκ, (1 − μ)κ); score = ∂/∂μ log p(b) = κ·[ln b − ln(1 − b) − ψ(μκ) + ψ((1 − μ)κ)], con ψ la función digamma. La corriente aplicada se recorta a [0.02, 1] por el mismo límite físico, pero el score se evalúa en la muestra sin recortar, que es la que salió de la política. μ se limita a [0.02, 0.98] para que los dos parámetros de la Beta sean positivos. En la forma B (la del profesor) el gradiente se evalúa en G = Gbest/Isc, recortado a [0.02, 0.98] para que ln(1 − G) sea finito.

Cómo hice justa la comparación. (1) Misma dispersión: para cada σ de la gaussiana, la Beta usa κ = 0.25/σ² − 1, de modo que ambas tienen la misma desviación estándar en μ = 0.5 (Figura K). Con esa elección el score de la Beta tiene la misma escala que el de la gaussiana en el centro, así que las tasas de aprendizaje son comparables. (2) Misma malla para las dos: 6 formas/baselines × 5 tasas × 4 σ × 2 normalizaciones = 240 configuraciones por

distribución. σ ∈ {0.05, 0.1, 0.2, 0.3} une la malla de la V2 (0.3) con su extensión (Paso 9); σ = 0.5 y 0.8 no se incluyen porque una distribución en [0, 1] no puede tener desviación ≥ 0.5 (y en la V2 ya eran peores para la gaussiana). (3) Mismo protocolo en dos etapas que el Paso 5: selección con las sombras sintéticas de entrenamiento de la V2 (semilla 7, 30 pares × 2 repeticiones) y evaluación una sola vez en los conjuntos de prueba (4 estáticos y 12 pares reales con 5 repeticiones; 100 sintéticas nuevas con 2), con el MSAPSO fiel en los mismos escenarios. (4) Diferencias pareadas por escenario con IC95 % bootstrap.

Consecuencias de la parametrización que hay que tener presentes. Con κ fijo la Beta no es una gaussiana «sin recorte»: con σ = 0.3 (κ = 1.78) es en forma de U en μ = 0.5 (pone la masa en los extremos) y con σ = 0.2 se vuelve muy asimétrica cerca de los bordes. Además, la desviación de la Beta depende de μ (en μ = 0.9 es 60 % de la de la gaussiana con el mismo σ). Ambas cosas son propiedades de la distribución, no errores, pero cambian lo que significa «σ» entre familias.

Corrección de reproducibilidad encontrada en el camino. La etapa 2 de la V2 sembraba cada escenario con `hash(nombre)`, que Python aleatoriza en cada ejecución (PYTHONHASHSEED); por eso sus cifras de prueba no se reproducen bit a bit. En la V3 la semilla usa `zlib.crc32(nombre)`, que es determinista. La etapa 1 de la V2 no tenía este problema y sí se reproduce exactamente.

Entorno de ejecución. Esta corrida se hizo en un entorno sin numba; el núcleo RK4 del convertidor se portó línea a línea a C (solo para correr aquí; no forma parte del repositorio). Validación: reproduce exactamente las cifras de la V2 para el MSAPSO fiel (97.26 / 96.41 / 96.04 % de energía; 100 / 100 / 99.23 % de potencia final) y la puntuación de la etapa 1 de la V2 para una configuración de control (84.12338648 %). En el repositorio el código corre con numba como el resto de la V2.

Archivos. `pvsim_V2/reinforce_dist_V3.py`, `reinforce_dist_grid_V3.py`, `graficas_dist_V3.py`, `resultados_V3/fig_densidades_gauss_beta.png`.

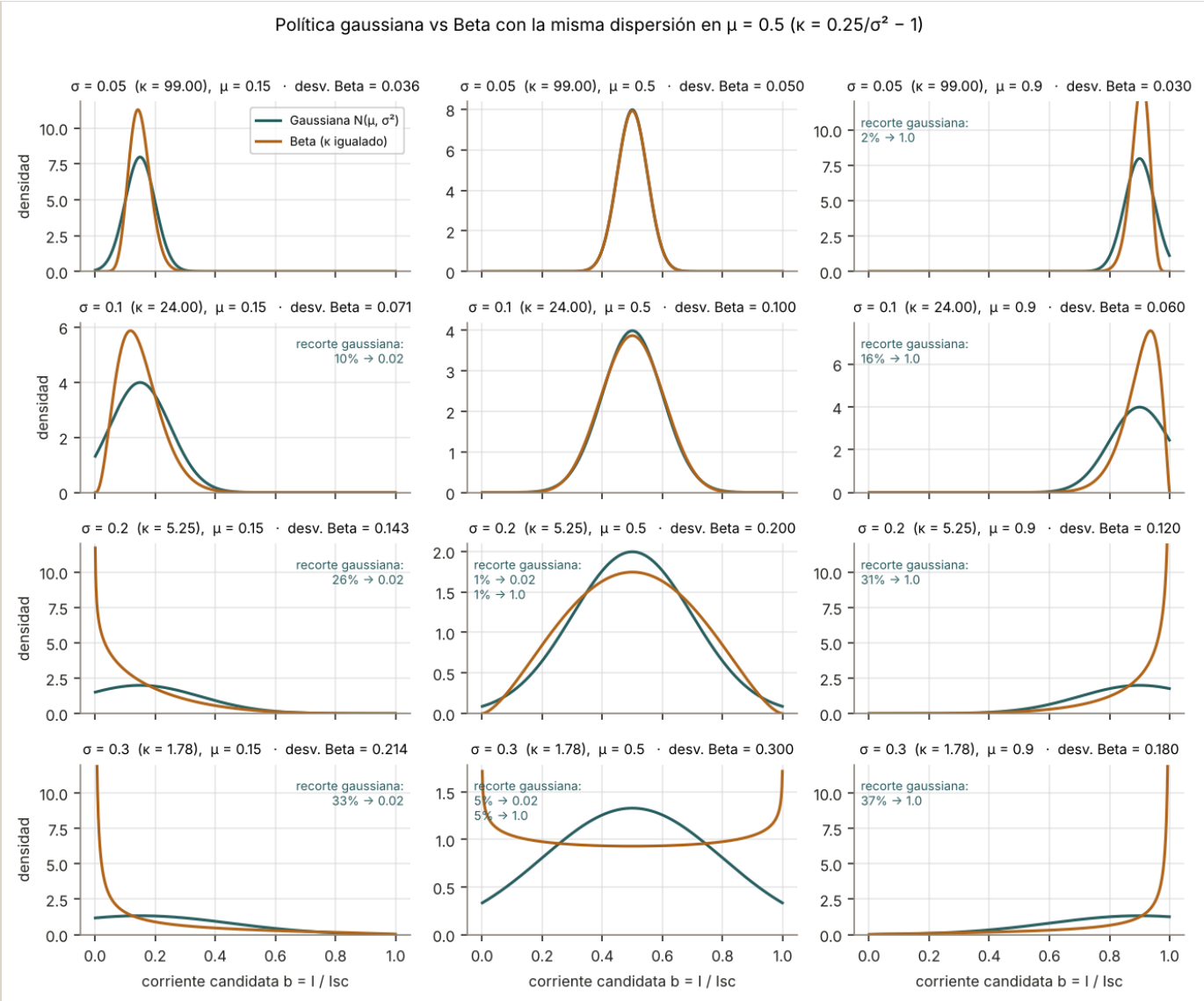


Figura K. Las dos políticas con la misma dispersión en μ = 0.5 (κ = 0.25/σ² − 1), para tres posiciones de μ. La gaussiana pierde masa por el recorte a [0.02, 1] cerca de los extremos; la Beta no sale de (0, 1) y su dispersión se encoge cerca de los bordes. Con σ = 0.2–0.3 la Beta se vuelve muy asimétrica (o en forma de U con σ = 0.3).

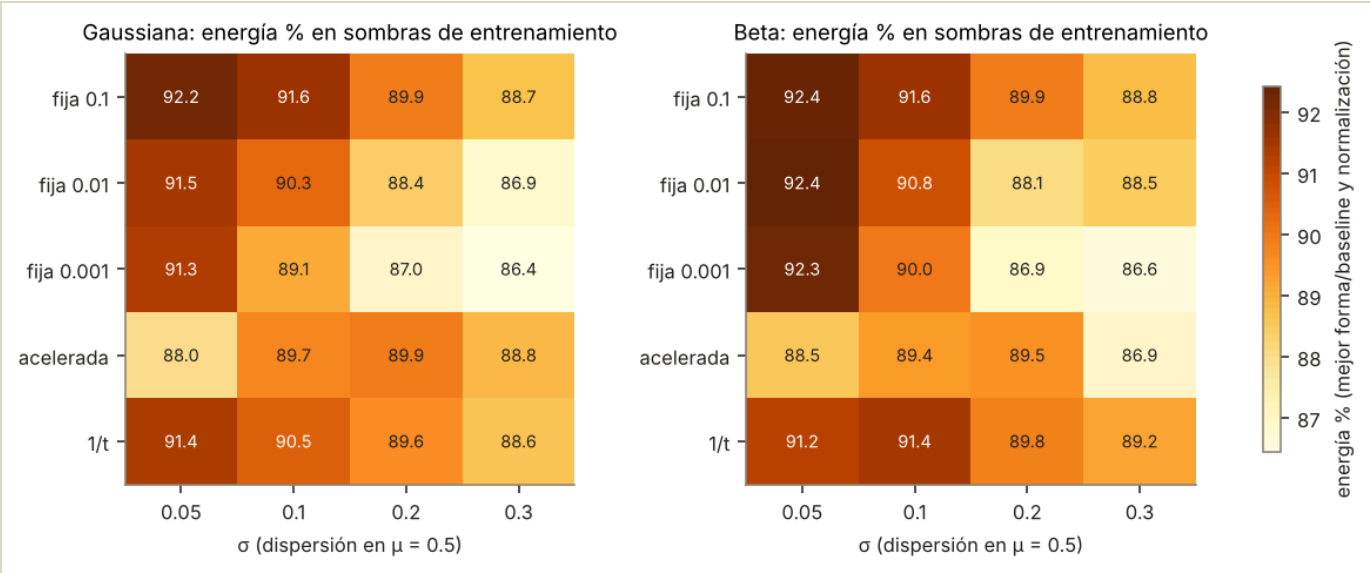


Figura L. Etapa 1 de la malla: energía % en las sombras sintéticas de entrenamiento para cada tasa de aprendizaje y σ (mejor forma/baseline y normalización de la recompensa en cada celda), para cada distribución.

PASO 14

Paso 14 — Resultados: gaussiana vs Beta en REINFORCE

Etapa 1 (selección, sombras de entrenamiento). Las dos familias eligen dispersión pequeña: σ = 0.05 en 9 de las 12 combinaciones y σ = 0.1 en las otras 3 (Figura L). La mejor energía de entrenamiento por σ es prácticamente la misma en ambas: 92.18 vs 92.43 % (σ = 0.05), 91.60 vs 91.60 % (0.1), 89.94 vs 89.94 % (0.2) y 88.82 vs 89.18 % (0.3), gaussiana vs Beta. En 4 de los 6 brazos (B sin baseline, B-V, A sin baseline, A-R−Q̂) la etapa 1 eligió exactamente los mismos hiperparámetros para las dos distribuciones, así que en esos brazos la diferencia de prueba se debe solo a la distribución. En A-R−V y A-Q̂−V eligió σ distinta (0.05 gaussiana, 0.1 Beta), así que ahí la diferencia mezcla distribución y σ.

Resultado en prueba (t_wait = 5 ms; energía % / potencia final %; estáticos · pares reales · sintéticos; `resultados_V3/analisis_dist_V3.txt`):

CONFIGURACIÓN (ELEGIDA EN LA ETAPA 1)	4 ESTÁTICOS	12 PARES REALES	100 SINTÉTICOS
MSAPSO fiel (referencia)	97.26 / 100.0	96.41 / 100.0	96.04 / 99.2
Gaussiana — B, sin baseline (lr 0.001, σ 0.05)	95.56 / 100.0	93.69 / 100.0	90.10 / 95.9
Beta — B, sin baseline (lr 0.001, σ 0.05)	**96.27 / 100.0**	**94.14 / 100.0**	**90.40 / 95.4**
Gaussiana — B, baseline V (lr 0.01, σ 0.05)	95.83 / 100.0	93.97 / 100.0	90.13 / 95.9
Beta — B, baseline V (lr 0.01, σ 0.05)	96.05 / 100.0	93.83 / 100.0	90.27 / 95.8
Gaussiana — A, sin baseline (lr 0.01, σ 0.1)	89.46 / 99.6	88.78 / 97.9	88.82 / 97.1
Beta — A, sin baseline (lr 0.01, σ 0.1)	89.86 / 99.5	88.30 / 97.5	89.72 / 97.0
Gaussiana — A, R−V (lr 0.1, σ 0.05)	92.03 / 99.8	92.51 / 99.0	89.62 / 96.9
Beta — A, R−V (lr 0.1, σ 0.1)	91.59 / 98.8	91.93 / 99.5	91.97 / 97.8
Gaussiana — A, R−Q̂ (lr 0.1, σ 0.05)	92.09 / 99.8	91.73 / 98.4	90.25 / 96.8
Beta — A, R−Q̂ (lr 0.1, σ 0.05)	90.77 / 99.7	91.47 / 98.3	90.05 / 96.6
Gaussiana — A, Q̂−V (1/t, σ 0.05)	90.78 / 99.8	92.17 / 99.9	90.59 / 97.0
Beta — A, Q̂−V (1/t, σ 0.1)	91.91 / 99.8	91.25 / 99.3	90.64 / 98.0

Control de consistencia con la V2. La configuración gaussiana B sin baseline elegida aquí es la misma que la mejor de la extensión del Paso 9 (lr 0.001, σ 0.05, R normalizada por la mejor potencia). La V2 reportó 95.55 · 93.85 · 89.94 % y aquí sale 95.56 · 93.69 · 90.10 %; la diferencia (≤ 0.16 puntos) es la esperada por el cambio de semillas de la etapa 2 (Paso 13).

Diferencia pareada Beta – gaussiana (puntos de energía; IC95 % bootstrap sobre escenarios; Beta mejor en n/N escenarios):

BRAZO	4 ESTÁTICOS	12 PARES REALES	100 SINTÉTICOS
B, sin baseline (mismos hiperparámetros)	**+0.72 [+0.31, +1.09] (4/4)**	**+0.45 [+0.27, +0.65] (11/12)**	+0.30 [−0.18, +0.74] (64/100)
B, baseline V (mismos)	+0.23 [−0.05, +0.60] (3/4)	−0.15 [−0.69, +0.32] (6/12)	+0.14 [−0.38, +0.68] (57/100)
A, sin baseline (mismos)	+0.40 [−0.51, +1.29] (3/4)	−0.48 [−1.43, +0.52] (5/12)	+0.90 [−0.33, +2.19] (54/100)
A, R–V (σ distinta)	−0.44 [−3.30, +1.48] (3/4)	−0.57 [−1.92, +1.01] (5/12)	+2.35 [+1.27, +3.50] (48/100)
A, R–Q̂ (mismos)	−1.32 [−4.34, +0.48] (2/4)	−0.26 [−1.12, +0.56] (7/12)	−0.20 [−1.22, +0.74] (45/100)
A, Q̂–V (σ distinta)	+1.13 [+0.10, +2.15] (4/4)	−0.92 [−2.34, +0.58] (4/12)	+0.05 [−1.04, +1.23] (37/100)

Lectura.

1. **Cambiar la distribución no cambia la conclusión de la V2.** Ninguna variante, con ninguna de las dos distribuciones, alcanza al MSAPSO fiel en transiciones de sombra. La mejor de todas (Beta, forma B sin baseline) queda a −2.28 puntos [−2.80, −1.66] en los pares reales y a −5.64 [−6.95, −4.48] en los sintéticos; en estáticos, −0.99 [−3.01, +1.04], sin diferencia demostrable.
2. **La forma B del profesor sigue siendo la mejor**, con las dos distribuciones (93.7–94.1 % en pares reales frente a 88.3–92.5 % de la forma A).
3. **El único efecto consistente de la Beta aparece en la forma B sin baseline**, donde la comparación es limpia (mismos hiperparámetros): +0.72 puntos en estáticos (4/4 casos) y +0.45 en pares reales (11/12). Es una mejora pequeña pero con intervalo que no incluye cero. En las sintéticas no es demostrable. En los demás brazos las diferencias están dentro del ruido o cambian de signo entre conjuntos. El +2.35 de A-R−V en sintéticas no se puede atribuir a la distribución: ahí la Beta usa σ = 0.1 y la gaussiana 0.05, y la Beta gana solo en 48 de 100 escenarios (la media la mueven pocos escenarios). Con 18 comparaciones al 95 % es esperable aproximadamente un falso positivo, así que los efectos aislados deben leerse con cautela.
4. **Interpretación (no probada causalmente): la ventaja de la Beta está en los bordes.** En la forma B sin baseline, la mayor ganancia por escenario se da en los casos cuyo óptimo alcanzable está cerca de b = 1: Caso 1 (b ≈ 0.94; +0.99 puntos) y Caso 3 (b ≈ 0.95; +1.18), y en las transiciones que terminan en ellos (2→3 +1.22; 3→1 +0.87). En el Caso 4 (óptimo en b ≈ 0.36) la diferencia es +0.09. Es lo que se espera si el beneficio viene de no recortar candidatos contra el límite superior. En esos dos casos la Beta incluso supera al MSAPSO fiel (99.01 vs 97.79 % y 99.00 vs 98.15 %). Son cuatro curvas, así que es una pista, no una conclusión.
5. **Por qué la diferencia es pequeña.** La selección empuja a σ = 0.05–0.1, y con dispersión tan pequeña las dos distribuciones son casi idénticas lejos de los bordes (Figura K, fila superior). En el par 2→4 la desviación de la Beta pasa solo de 0.050 a 0.048 (Figura N). La

desventaja frente al MSAPSO no es la forma de la distribución sino la dispersión fija: durante las 25 pasadas de búsqueda los candidatos siguen dispersos y la potencia oscila entre 50 y 100 %, mientras el MSAPSO contrae el enjambre hacia la pasada 12 (Figura N, panel inferior). Es la misma explicación del Paso 5.

6. **Hallazgo lateral (Figura N).** En el par 2→4, las dos políticas terminan con μ ≈ 0.58 y no en el óptimo marcado (0.36): en el Caso 4 hay dos picos casi empatados (75.15 W en b ≈ 0.58 y 75.19 W en b ≈ 0.36, diferencia 0.06 %). La métrica de energía no distingue entre ellos; coincide con lo encontrado en el análisis exploratorio del Avance 1.

Siguiente paso natural. Dejar que la política aprenda también su dispersión: σ (gaussiana) o κ (Beta) como segundo parámetro con su propio gradiente, o un calendario decreciente. Con dispersión adaptativa la Beta tiene una ventaja estructural (su varianza ya depende de μ y está acotada), y es donde la diferencia entre familias podría importar. También queda pendiente PPO con política Beta (requiere una distribución propia en stable-baselines3).

Archivos. `reinforce_dist_grid_V3.py`, `analisis_dist_V3.py`, `graficas_dist_V3.py`, `resultados_V3/reinforce_dist_grid_V3.json`, `resultados_V3/analisis_dist_V3.txt|json`, `resultados_V3/fig_*.png`.

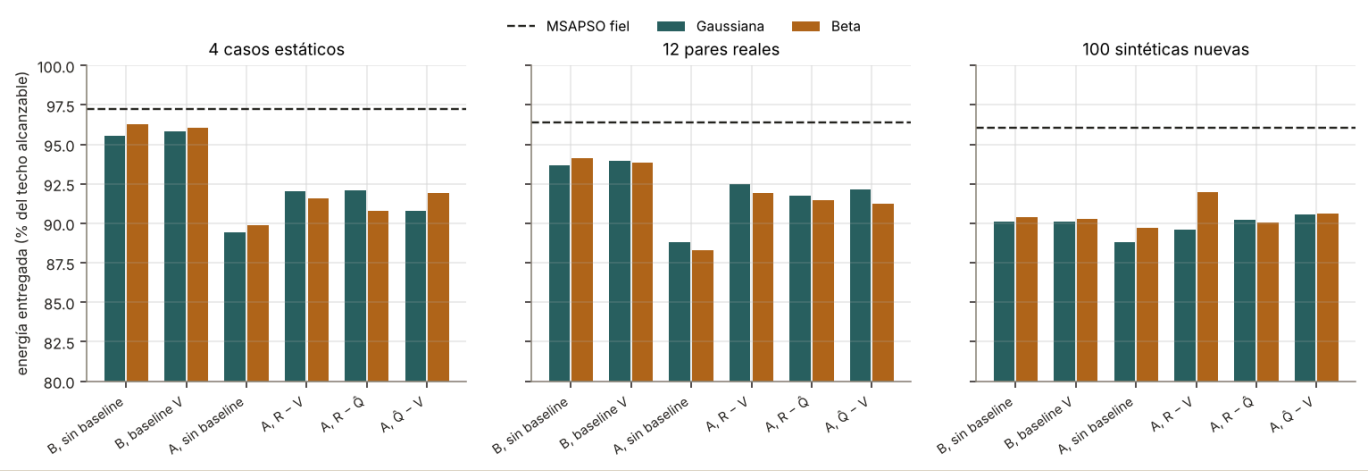


Figura M. Energía entregada en los conjuntos de prueba por forma/baseline y distribución (configuración elegida en la etapa 1). Línea punteada = MSAPSO fiel.

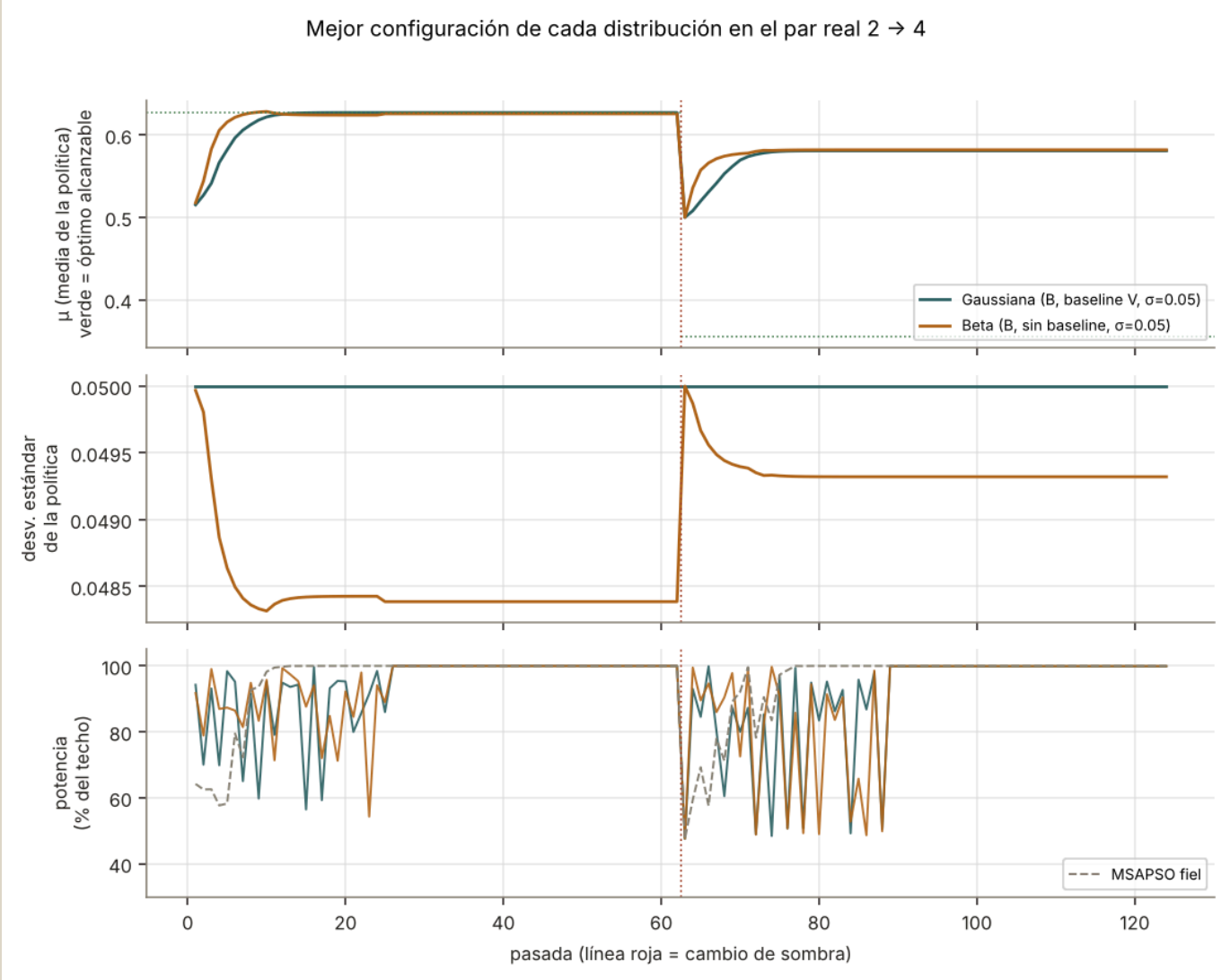


Figura N. Qué hace la mejor configuración de cada distribución en un par real: media μ de la política, su desviación estándar y la potencia por pasada (misma semilla de escenario).

5. Conclusiones

1. **Retractación.** La conclusión de la V1 («RL-MSAPSO supera al MSAPSO: 82.85 % → 99.33 % en transiciones») no se sostiene. Era un artefacto de un baseline que no era el MSAPSO del paper, de una observación con información privilegiada y de una métrica inadecuada.
2. **Con comparación fiel y justa** (mismo simulador, energía entregada, 3 semillas), PPO supera al MSAPSO del paper únicamente en sombras estáticas (+1.84 puntos de energía, IC95 % [+1.23, +2.42]); en las 12 transiciones reales (+0.42, IC [−1.35, +1.88]) y en 100 transiciones sintéticas (−0.82, IC [−2.04, +0.34]) no hay diferencia demostrable, y en el régimen de 0.5 ms tampoco.
3. **La ganancia en estáticos es un calendario, no aprendizaje.** El agente aprendió a colapsar el enjambre desde la 1.ª pasada (β ≈ 0.95). Una regla fija equivalente (β = 0.95, t_max = 5) lo iguala o supera en casos reales; pero esa regla es mala con el convertidor lento de 0.5 ms y no gana en sombras sintéticas. La conveniencia de un calendario agresivo depende de la distribución de sombras. Por qué favorece a los 4 casos reales y no a las sintéticas no lo investigué a fondo: no es simplemente un efecto de la rejilla inicial (su mejor punto alcanza solo 85.5 % y 91.0 % del techo en los casos 2 y 4); probablemente influye que, con el convertidor dinámico, las mediciones recorren la trayectoria entre referencias. Queda como pregunta abierta.
4. **Lo que sí demuestra el trabajo:** (a) el MSAPSO fiel es robusto (100 % de potencia final con 5 ms; el reinicio de 5 W es esencial: sin él, 56.6 % de energía en transiciones); (b) el criterio de comparación importa más que el algoritmo: la misma pregunta dio respuestas opuestas con un baseline débil y con uno fiel.
5. **REINFORCE gaussiano** funciona como método de búsqueda (potencia final ≈ 100 %), pero con σ fija desperdicia energía explorando y no alcanza al MSAPSO (95.6 / 93.9 / 89.9 % con σ = 0.05, la mejor variante). Un σ decreciente es el siguiente paso natural.
6. **Gaussiana vs Beta (V3).** Cambiar la familia de la política no cierra la brecha con el MSAPSO: la causa es la dispersión fija (se sigue explorando durante toda la búsqueda), no la forma de la distribución. La Beta da una mejora pequeña y demostrable solo en la forma B sin baseline (+0.72 puntos en estáticos, +0.45 en pares reales), concentrada en los casos con el óptimo cerca del límite de corriente, donde la gaussiana recorta candidatos.

Recomendaciones para seguir

- Que el profesor **valide la especificación del MSAPSO** (Anexo) y responda las preguntas de la Sección 6; con eso se puede re-correr todo cambiando parámetros, no código.
- **Repetir en los simuladores del profesor** (convertidor que asiente a 0.5 ms): es donde la comparación sería más cercana al paper. La conclusión aquí es condicional a nuestro simulador.

- Evaluar donde RL podría aportar más: **ruido de medición, parámetros desconocidos o cambiantes, patrones de sombra con picos estrechos que la rejilla inicial falle**, y objetivos distintos (p. ej., minimizar la energía perdida durante la búsqueda con β/t_max/reinicio aprendidos de forma conjunta).
- Ampliar el conjunto de transiciones reales (con más de 4 casos) y repetir con más semillas; probar REINFORCE con σ decreciente y el brazo «tasa por PPO».
- **V3:** aprender la dispersión de la política (σ o κ) junto con μ y comparar de nuevo gaussiana vs Beta; implementar PPO con política Beta (distribución propia en stable-baselines3).

6. Limitaciones, supuestos declarados y preguntas para el profesor

Limitaciones y supuestos declarados

- **Simulador propio.** No es el de la versión final del paper. El modelo del Boost es promediado (sin PWM), el sensor es ideal (sin ruido), y con L = 10 mH el convertidor simulado no sigue referencias de 0.5 ms (Pasos 3). El régimen principal (5 ms) es una decisión mía, declarada; el literal de 0.5 ms se reporta aparte.
- **Datos.** Las 12 transiciones «reales» son combinaciones de los 4 casos del paper hechas por mí (cambio a la mitad del episodio); las 100 transiciones sintéticas vienen de un generador propio (modelo de un diodo por sub-string con bypass) no validado contra mediciones.
- **Fidelidad del MSAPSO.** Leída de la Fig. 6 y de las indicaciones del profesor. Decisiones no cubiertas: sentido de la 1.ª pasada (menor→mayor; el orden literal de la figura se evaluó como sensibilidad: a 5 ms 10/12 transiciones ≥ 99 % en vez de 12/12), Eq. 13 sin ε, reinicio solo tras t_max con umbral absoluto de 5 W, Isc re-medido sin costo en cada reinicio, sensor en los terminales del panel.
- **RL.** Hiperparámetros por defecto de stable-baselines3 (sin ajuste fino), 3 semillas para PPO principal, 2 para ablaciones y 3 M, *una* para A2C y SAC. La desviación entre semillas (≈ 1 punto) es comparable a varias diferencias reportadas.
- **Control sin RL.** La búsqueda formal (dos etapas) es honesta; las dos «reglas simples» se eligieron *después* de ver al agente y evaluarlas en los mismos conjuntos de prueba, por lo que son ilustrativas, no un resultado de selección limpia.
- **Métrica.** La energía entregada depende del horizonte (62 pasadas por segmento): un horizonte más largo diluye el costo de la búsqueda y acerca a todos los métodos.
- **APSO (Fase 0).** α y β no vienen en el paper (se barrió $\alpha \in \{0.05, 0.1, 0.2\}$, $\beta = 0.4$); y la Fase 0 es sobre curvas estáticas.
- **REINFORCE.** σ fija (lo propuesto) y una extensión con σ pequeña; sin σ decreciente; sin el brazo «tasa por PPO».
- **Gaussiana vs Beta (V3).** La equivalencia entre familias se fijó igualando la desviación en $\mu = 0.5$; con κ fijo la dispersión de la Beta depende de μ, así que fuera del centro no exploran lo mismo. Solo REINFORCE (no PPO), solo t_wait = 5 ms y $\sigma < 0.5$. Se hicieron 18 comparaciones pareadas al 95 %, así que es esperable aproximadamente un falso positivo; el efecto de la forma B es el único consistente en dos conjuntos. Corrida en un entorno sin numba con el núcleo RK4 portado a C, validado contra las cifras exactas de la V2.

Preguntas para el profesor (no bloquean nada; si cambian una decisión se re-corre cambiando un parámetro)

1. En su simulador, ¿el convertidor asienta en 0.5 ms? ¿Con qué L, ganancias y carga? En el nuestro, con L = 10 mH y las ganancias del paper, la corriente del inductor se mueve ≈ 0.6 A por 0.5 ms y no alcanza.
2. ¿La primera pasada del MSAPSO va de menor a mayor corriente (como dijo) o en el orden de la lista de la Fig. 6 (descendente)? Con la segunda lectura el Caso 4 mejora de 97.5 % a 99.8 % a 0.5 ms.
3. La Fig. 6 cita las Eq. (12) y (13) en la actualización de posiciones: ¿el MSAPSO lleva el término ε o es la Eq. (13) determinista?
4. El reinicio solo se evalúa tras t_max (con el convertidor fijo en Gbest): ¿es intencional que no haya detección durante la búsqueda?
5. ¿Isc se vuelve a medir en cada reinicio, y a qué costo de energía?
6. Para REINFORCE: ¿la forma principal es la suya, $(Gbest - \mu)\cdot R/\sigma^2$, o el estimador de libro con las muestras? ¿Quiere que se pruebe un σ decreciente?
7. Para la Beta (V3): ¿la quiere con concentración fija (como aquí, igualada a la σ de la gaussiana) o aprendiendo ambos parámetros (α, β)? ¿Quiere también PPO con política Beta?
8. Para APSO: ¿valores de α y β que usó el paper (no aparecen en el borrador)?

7. Mapa de archivos y cómo reproducir

Todo vive en `Proyecto Integrador/Proyecto Integrador Paneles_V2/` (la V1 en `Proyecto Integrador Paneles/` no se modificó). Todos los archivos nuevos terminan en `_V2`. La V3 (pasos 13–14) solo **agrega** archivos con sufijo `_V3` en la misma carpeta; no modifica ningún archivo de la V2.

ARCHIVO	QUÉ CONTIENE / QUÉ PRODUCE
`bitacora_V2.md`	Registro paso a paso con el razonamiento (fuente de este PDF).
`pvsim_V2/pvcurve_V2.py`, `array_model_V2.py`	Curvas de los 4 casos reales (con P_ceiling) y generador de curvas sintéticas.
`pvsim_V2/converter_V2.py`	Modelo dinámico PV + Boost + Super-Twisting (RK4 compilado con numba).
`pvsim_V2/msapso_V2.py`	MSAPSO fiel (Fig. 6), ejecutor de escenarios y métricas.
`pvsim_V2/passrunner_V2.py`, `gym_env_V2.py`	Mecánica de una pasada y entorno Gymnasium del agente (sin información privilegiada).
`pvsim_V2/reinforce_V2.py`	Rama REINFORCE gaussiano (formas A y B, baselines R–V, R–Q̂, Q̂–V).
`pvsim_V2/harness_V2.py`	Arnés común de evaluación (casos estáticos, pares reales, sintéticos).
`validar_simulador_V2.py`	Equivalencia del simulador V2 con la V1 y velocidad (Paso 1).
`evaluar_baselines_V2.py` → `resultados_V2/baselines_V2.txt|json`	MSAPSO fiel y variantes en estáticos y 12 transiciones, a 5 y 0.5 ms (Pasos 2–3).
`resintonia_controlador_V2.py`	Prueba de re-sintonía del Super-Twisting (Paso 3).
`optimizar_msapso_V2.py` → `msapso_optimizado_5ms.json`	Control sin RL con selección en dos etapas (Paso 7).
`reinforce_grid_V2.py` → `reinforce_grid_V2.json`, `reinforce_grid_ext_V2.json`	Malla de REINFORCE y extensión con σ pequeña (Pasos 5 y 9).
`train_V2.py`, `lanzar_entrenamientos_V2.sh`, `lanzar_extension_V2.sh` → `modelos_V2/`	Entrenamiento de PPO/A2C/SAC (3 semillas × 2 regímenes, ablaciones, extensión a 3 M).
`evaluar_rl_V2.py`, `analisis_final_V2.py`, `curva_aprendizaje_V2.py` → `resultados_V2/*.json|txt`	Evaluaciones y estadística consolidada.
`graficas_V2.py`, `graficas_resultados_V2.py` → `resultados_V2/*.png`	Todas las figuras.
`construir_reporte_V2.py`	Genera este PDF.
`pvsim_V2/reinforce_dist_V3.py`	V3: REINFORCE con política gaussiana (idéntica a la V2) o Beta (media-concentración, $\kappa = 0.25/\sigma^2 - 1$).

ARCHIVO	QUÉ CONTIENE / QUÉ PRODUCE
`reinforce_dist_grid_V3.py` → `resultados_V3/reinforce_dist_grid_V3.json`	V3: malla en dos etapas gaussiana vs Beta (480 configuraciones) y evaluación en prueba con el MSAPSO fiel (Pasos 13–14).
`analisis_dist_V3.py` → `resultados_V3/analisis_dist_V3.txt|json`	V3: tablas y diferencias pareadas Beta − gaussiana con IC95 % bootstrap.
`graficas_dist_V3.py` → `resultados_V3/*.png`	V3: figuras K–N.
`bitacora_V3.md` , `reporte_frag_V3/` , `construir_reporte_V3.py`	V3: bitácora V2 + pasos 13–14 y fuentes de este PDF.

Cómo reproducir (desde la carpeta V2, con las dependencias de `requirements_V2.txt`)

```
python3 validar_simulador_V2.py                    # Paso 1: simulador V2 vs V1
python3 evaluar_baselines_V2.py                    # Pasos 2-3: MSAPSO fiel y variantes
python3 train_V2.py --wait 5e-3 --seed 0 --timesteps 1000000 --n-envs 4 --tag ppo_5ms_full_s0     # entrena un PPO (~8-15 min)
python3 analisis_final_V2.py                       # estadistica consolidada (usa cache en resultados_V2/raw/)
python3 graficas_resultados_V2.py                  # figuras
python3 construir_reporte_V2.py                    # PDF de la V2
# --- V3: gaussiana vs Beta ---
python3 reinforce_dist_grid_V3.py --procs 4        # malla (~40 min con 2 nucleos)
python3 analisis_dist_V3.py                        # tablas y diferencias pareadas
python3 graficas_dist_V3.py                        # figuras K-N
python3 construir_reporte_V3.py                    # este PDF
```

ANEXO

Anexo. Código clave y especificación del MSAPSO implementado

Especificación del MSAPSO tal como está implementado (lectura de la Fig. 6)

Parámetros: β_min = 0.1, β_max = 0.95, t_max = 25, t_wait ∈ {0.5 ms, 5 ms}, 7 partículas con posiciones iniciales x_i = Isc·[0.95, 0.8, 0.65, 0.5, 0.3, 0.15, 0.1] (corrientes de referencia), umbral de reinicio = 5 W.

```
INICIO / REINICIO:  Pbest = 0 ; Gbest = 0 ; x = Isc * [0.95 ... 0.10] (Isc medido) ; t = 1 ; sentido = ascendente
REPETIR mientras t <= t_max:
    PASADA (conjunto de 7 valores, en sentido ascendente o descendente de corriente, alternando):
        PARA cada particula i en ese orden:
            aplicar I_ref = x_i al convertidor durante t_wait ; medir Vpv e Ipv (corriente de salida del modulo)
            Ppv = Vpv * Ipv
            SI Ppv >= Pbest:  Pbest = Ppv ; Gbest = Ipv          (actualizacion inmediata, dentro de la pasada)
    beta_t = beta_min + (t / t_max) * (beta_max - beta_min)          (Eq. 14)
    x = x + beta_t * (Gbest - x)  (todas las particulas)           (Eq. 13, determinista)
    invertir el sentido del siguiente barrido ; t = t + 1
MANTENIMIENTO ("Set Gbest, Measure power"):  I_ref = Gbest ; medir Ppv
    SI |Pbest - Ppv| >= 5 W:  REINICIO (volver a INICIO)
```

Decisiones de implementación que NO están en el paper (declaradas)

- **Sentido de la 1.ª pasada:** de menor a mayor corriente (indicación del profesor). La lista de la Fig. 6 está en orden descendente; la variante «1.ª pasada descendente» se evaluó como sensibilidad (Paso 3).
- **Eq. 12 o Eq. 13:** la figura cita ambas; se usó la Eq. 13 determinista (el paper presenta MSAPSO como determinista).
- **Reinicio solo tras t_max** y con umbral absoluto de 5 W, como en la figura. Una variante de «detección temprana» se probó y se descartó (Paso 3).
- **Isc se vuelve a medir en cada reinicio** (la figura siembra con Isc; se asume disponible sin costo de energía).
- **Una pasada de mantenimiento** dura lo mismo que una de barrido (7 ventanas de t_wait), para comparar energía en la misma base temporal.
- **Sensor:** Vpv e Ipv medidos en los terminales del panel (Ipv = I_panel(Vpv)); la energía se contabiliza como la que entrega el panel.

pvsim_V2/msapso_V2.py — 175 LÍNEAS

```
"""
msapso_V2 — MSAPSO_I FIEL a la Figura 6 del paper + ejecutor de escenarios y metricas.

Especificacion (leida de la Fig. 6 como imagen, p. 11 del paper; aclaraciones del profesor):
  * Valores:  beta_min=0.1, beta_max=0.95, t_max=25, t_wait=0.5 ms
  * 7 particulas, posiciones iniciales  x_i = Isc * [0.95 0.8 0.65 0.5 0.3 0.15 0.1]   (corrientes de referencia)
  * "Pasada positiva" y "pasada negativa" ALTERNADAS: un conjunto de valores se prueba de menor a mayor corriente,
    el siguiente de mayor a menor, y asi sucesivamente (escalera; evita el salto grande de regreso).
  * Cada candidato se aplica al convertidor durante t_wait y se mide Ppv. Si Ppv >= Pbest -> Pbest=Ppv, Gbest=Ipv
    (Gbest se actualiza AL INSTANTE, dentro del barrido; Ipv = corriente medida).
  * Al terminar cada pasada las posiciones se recalculan con la Eq.(13) determinista:  x <- x + beta*(Gbest - x),
    con beta de la Eq.(14): beta_t = beta_min + (t/t_max)(beta_max - beta_min).
```

```
  * Tras t_max pasadas: el convertidor se fija en Gbest ("park") y se mide Ppv; si |Pbest - Ppv| >= 5 W se REINICIA
    todo (Pbest=0, Gbest=0, posiciones iniciales, t=1).
Variantes para comparacion (declaradas en la bitacora): restart_mode = "paper" | "early" | "none";
  first_pass = "ascending" (dicho por el profesor) | "descending" (orden literal de la lista en la Fig. 6).

Unidad de tiempo del entorno: un "paso" = 7 ventanas de t_wait (el tiempo de una pasada completa). Una pasada de barrido
y un paso de mantenimiento (park) duran lo mismo, de modo que la energia de todos los metodos se compara en la misma base.
"""
import numpy as np
from pvsim_V2.converter_V2 import Plant

INIT_FRACS = (0.95, 0.80, 0.65, 0.50, 0.30, 0.15, 0.10)
N_PART = 7


class Swarm:
    """Estado del enjambre y operaciones elementales."""

    def __init__(self, first_pass="ascending", fracs=INIT_FRACS, gbest_from="measured", alternate=True):
        self.alternate = alternate            # False = siempre ascendente (como la V1; solo para ablacion)
        self.first_pass = first_pass
        self.fracs = np.array(sorted(fracs), dtype=float)   # ascendente; el orden de evaluacion lo fija `direction`
        self.gbest_from = gbest_from
        self.n = len(fracs)
        self.n_restarts = 0
        self.Isc = 1.0
        self.restart(1.0, first=True)

    def restart(self, Isc, first=False):
        """Bloque &#x27;Initial Value / Set Initial Particle Position' de la Fig. 6."""
        self.Isc = float(Isc)                 # el algoritmo siembra con Isc (se re-mide al reiniciar)
        self.x = self.Isc * self.fracs.copy()
        self.Pbest = 0.0
        self.Gbest = 0.0
        self.direction = self.first_pass
        self.t = 1                            # contador de pasadas desde el (re)inicio
        if not first:
            self.n_restarts += 1

    def sweep(self, plant, wait):
        """Una pasada: cada particula se aplica durante `wait`, de menor a mayor o de mayor a menor corriente."""
        order = np.argsort(self.x)
        if self.direction == "descending":
            order = order[::-1]
        E, Ps = 0.0, []
        for i in order:
            P, e = plant.run(wait, self.x[i])
            E += e
            Ps.append(P)
            if P >= self.Pbest:               # "Ppv >= Pbest -> Pbest = Ppv, Gbest = Ipv"
                self.Pbest = P
                self.Gbest = plant.Ipv if self.gbest_from == "measured" else float(self.x[i])
        return E, np.array(Ps)

    def update(self, beta):
        """Eq.(13): x <- x + beta (Gbest - x). Despues alterna la direccion del siguiente barrido."""
        self.x = np.clip(self.x + beta * (self.Gbest - self.x), 0.0, self.Isc)
        if self.alternate:
            self.direction = "descending" if self.direction == "ascending" else "ascending"
        self.t += 1

    def hold(self, plant, wait, n=None):
        """Convertidor fijo en Gbest durante una pasada de tiempo (n ventanas). Devuelve (E, P_final)."""
        n = self.n if n is None else n
        P, E = plant.run(n * wait, self.Gbest)
        return E, P

    def dispersion(self):
        return float(np.std(self.x) / max(self.Isc, 1e-9))


class MSAPSOController:
    """Controlador MSAPSO_I (maquina de estados de la Fig. 6). `step(plant)` ejecuta un paso de tiempo."""

    def __init__(self, wait, beta_min=0.1, beta_max=0.95, t_max=25, restart_mode="paper", thresh=5.0,
                 first_pass="ascending", gbest_from="measured", beta_fn=None, alternate=True, fracs=INIT_FRACS):
        self.wait, self.beta_min, self.beta_max, self.t_max = wait, beta_min, beta_max, t_max
        self.restart_mode, self.thresh = restart_mode, thresh
        self.beta_fn = beta_fn                # opcional: reemplazar la Eq.(14) (p. ej. beta fijo, o beta aprendido)
        self.sw = Swarm(first_pass=first_pass, gbest_from=gbest_from, alternate=alternate, fracs=fracs)
        self.mode = "search"
        self.started = False

    def _beta(self):
        if self.beta_fn is not None:
            return self.beta_fn(self.sw.t)
        return self.beta_min + (self.sw.t / self.t_max) * (self.beta_max - self.beta_min)   # Eq.(14)

    def step(self, plant):
        sw = self.sw
        if not self.started:                  # primer arranque: Isc medido de la curva actual
```

```
            sw.restart(plant.curve.Isc, first=True)
            self.started = True
        restarted = False
        if self.mode == "search":
            beta = self._beta()
            E, Ps = sw.sweep(plant, self.wait)
            P_end, P_max = float(Ps[-1]), float(Ps.max())
            disp = sw.dispersion()
            t_now = sw.t
            sw.update(beta)
            if self.restart_mode == "early" and t_now > 1 and (sw.Pbest - P_max) >= self.thresh:
                sw.restart(plant.curve.Isc); restarted = True      # detecta el cambio ya durante la busqueda
            elif sw.t > self.t_max:
                self.mode = "park"            # "Set Gbest, Measure power"
            mode = "search"
        else:
            E, P_end = sw.hold(plant, self.wait)
            P_max, beta, disp, t_now, mode = P_end, 0.0, sw.dispersion(), sw.t, "park"
            if self.restart_mode != "none" and abs(sw.Pbest - P_end) >= self.thresh:   # |Pbest-Ppv| >= 5
                sw.restart(plant.curve.Isc); self.mode = "search"; restarted = True
        return dict(E=E, P_end=P_end, P_max=P_max, beta=beta, mode=mode, disp=disp, t=t_now,
                    Gbest=sw.Gbest, Pbest=sw.Pbest, restart=restarted)


# ---------------------------------------------------------------------------------------------
# Ejecutor de escenarios y metricas
# ---------------------------------------------------------------------------------------------
def run_scenario(make_controller, segments, wait, dt=1e-5):
    """segments = [(curva, duracion_s), ...]. El convertidor conserva su estado al cambiar de sombra.
    Devuelve un dict de arreglos (uno por paso de tiempo) y los datos por segmento."""
    plant = Plant(segments[0][0], dt=dt)
    ctrl = make_controller()
    step_T = ctrl.sw.n * wait
    rec = {k: [] for k in ("t0", "E", "P_end", "P_max", "beta", "mode", "disp", "tpass", "Gbest", "Pbest", "restart", "seg")}
    t = 0.0
    for si, (curve, dur) in enumerate(segments):
        plant.set_curve(curve)
        n_steps = int(round(dur / step_T))
        for _ in range(n_steps):
            info = ctrl.step(plant)
            rec["t0"].append(t); t += step_T
            for k, src in (("E", "E"), ("P_end", "P_end"), ("P_max", "P_max"), ("beta", "beta"), ("mode", "mode"),
                           ("disp", "disp"), ("tpass", "t"), ("Gbest", "Gbest"), ("Pbest", "Pbest"), ("restart", "restart")):
                rec[k].append(info[src])
            rec["seg"].append(si)
    log = {k: np.array(v) for k, v in rec.items()}
    log["step_T"] = step_T
    log["segments"] = segments
    return log


def segment_metrics(log, si):
    """Metricas de un segmento (una sombra). Todas relativas a P_ceiling (potencia alcanzable con R=12 ohm)."""
    curve, dur = log["segments"][si]
    m = log["seg"] == si
    Pref = curve.P_ceiling
    T = m.sum() * log["step_T"]
    E = log["E"][m].sum()
    P_end = log["P_end"][m]
    modes = log["mode"][m]
    n = len(P_end)
    tail = slice(int(0.9 * n), n)                              # ultimo 10 % del segmento
    park_idx = np.where(modes == "park")[0]
    ok99 = np.where(P_end >= 0.99 * Pref)[0]
    return dict(
        eff_energy=100.0 * E / (Pref * T),                     # energia entregada / energia ideal (metrica principal)
        eff_final=100.0 * float(np.mean(P_end[tail])) / Pref,  # potencia al final del segmento / ideal
        t_first_park=float(park_idx[0] * log["step_T"]) if len(park_idx) else float("nan"),   # fin de la 1.a busqueda
        t99=float(ok99[0] * log["step_T"]) if len(ok99) else float("nan"),                   # 1.a vez >= 99 % del techo
        n_restarts=int(log["restart"][m].sum()),
        eff_gmpp_energy=100.0 * E / (curve.P_gmpp * T),         # idem contra el GMPP teorico (para comparar con el paper)
    )
```

pvsim_V2/passrunner_V2.py

85 LÍNEAS

```
"""
passrunner_V2 — Mecanica de UNA pasada del enjambre, compartida por el entorno de RL y por los controladores
que se evaluan con el mismo arnes (una sola fuente de verdad: lo que aprende el agente y lo que se mide son identicos).

Observacion del agente (SOLO senales medibles en un controlador real; sin informacion privilegiada):
  La V1 normalizaba la potencia por el GMPP real (desconocido para un controlador) y avisaba directamente del cambio de
  sombra. Aqui NO: la potencia se expresa en unidades de 100 W (escala del panel) y el cambio de sombra solo se puede
  inferir de la potencia medida. El GMPP real y P_ceiling se usan UNICAMENTE en la recompensa durante el entrenamiento.
    f0  P_max_pasada / 100 W        mejor potencia medida en la ultima pasada
    f1  P_media_pasada / 100 W      potencia media de la ultima pasada
```

```
    f2  Pbest / 100 W               mejor potencia almacenada desde el (re)inicio
    f3  1 - P_max_pasada / Pbest    caida relativa respecto al mejor guardado (senal de cambio de sombra), en [0,1]
    f4  dispersion de particulas    std(x)/Isc  (cuan colapsado esta el enjambre = historial acumulado de beta)
    f5  pasadas desde el reinicio / 25  (recortado a 3)
    f6  beta usado en la pasada anterior (memoria propia del agente)
    f7  (Pbest - Pbest_anterior) / 100 W   cuanto subio Pbest en la ultima pasada
"""
import numpy as np
from pvsim_V2.msapso_V2 import Swarm

P_SCALE = 100.0
FEATURES = ["P_max", "P_media", "Pbest", "caida_rel", "dispersion", "t_reinicio", "beta_prev", "dPbest"]


class PassRunner:
    def __init__(self, wait, first_pass="ascending"):
        self.wait = wait
        self.sw = Swarm(first_pass=first_pass)
        self.started = False
        self.since_reset = 0
        self.last_beta = 0.5
        self.last = dict(P_max=0.0, P_mean=0.0, dPbest=0.0)

    def start(self, plant):
        self.sw.restart(plant.curve.Isc, first=True)
        self.started = True
        self.since_reset = 0
        self.last_beta = 0.5
        self.last = dict(P_max=0.0, P_mean=0.0, dPbest=0.0)

    def observe(self):
        sw, L = self.sw, self.last
        drop = 0.0 if sw.Pbest <= 1e-9 else float(np.clip(1.0 - L["P_max"] / sw.Pbest, 0.0, 1.0))
        return np.array([L["P_max"] / P_SCALE, L["P_mean"] / P_SCALE, sw.Pbest / P_SCALE, drop,
                         sw.dispersion(), min(self.since_reset / 25.0, 3.0), self.last_beta,
                         L["dPbest"] / P_SCALE], dtype=np.float32)

    def do_pass(self, plant, beta, reset):
        """Ejecuta una pasada (7 ventanas de `wait`) con el paso `beta` elegido y, opcionalmente, reiniciando antes."""
        if not self.started:
            self.start(plant)
        sw = self.sw
        if reset:
            sw.restart(plant.curve.Isc)
            self.since_reset = 0
        Pbest_before = sw.Pbest
        E, Ps = sw.sweep(plant, self.wait)
        disp = sw.dispersion()
        sw.update(beta)
        self.since_reset += 1
        self.last_beta = float(beta)
        self.last = dict(P_max=float(Ps.max()), P_mean=float(Ps.mean()), dPbest=float(sw.Pbest - Pbest_before))
        return dict(E=E, P_end=float(Ps[-1]), P_max=float(Ps.max()), beta=float(beta), mode="search", disp=disp,
                    t=self.since_reset, Gbest=sw.Gbest, Pbest=sw.Pbest, restart=bool(reset))


class PolicyController:
    """Adaptador: un &#x27;agente' (funcion obs -> (beta, reset)) con la misma interfaz .step(plant) que MSAPSOController,
    para evaluarlo con el MISMO arnes (run_scenario / segment_metrics) que los baselines."""

    def __init__(self, wait, act_fn, mask=None):
        self.runner = PassRunner(wait)
        self.act_fn = act_fn
        self.mask = mask                      # indices de features visibles (para la ablacion)
        self.sw = self.runner.sw

    def step(self, plant):
        r = self.runner
        if not r.started:
            r.start(plant)
        obs = r.observe()
        if self.mask is not None:
            obs = obs[self.mask]
        beta, reset = self.act_fn(obs)
        return r.do_pass(plant, beta, reset)
```

pvsim_V2/gym_env_V2.py

79 LÍNEAS

```
"""
gym_env_V2 — Entorno Gymnasium V2 para MPPT bajo sombreado parcial cambiante.

DIFERENCIAS CLAVE RESPECTO A LA V1 (cada una responde a una observacion del profesor o a una autocritica propia):
  1. La mecanica de una pasada es la del MSAPSO FIEL (7 particulas, barrido alternante, Gbest inmediato, Eq.13), compartida
     con los baselines via PassRunner => comparacion justa.
  2. Observacion SIN informacion privilegiada (ver passrunner_V2.py): sin normalizar por GMPP, sin aviso de cambio de sombra.
```

```
  3. Recompensa = ENERGIA entregada en la pasada / energia ideal (P_ceiling * duracion de la pasada). Premia exactamente lo que
     importa (energia cosechada) y castiga de forma natural explorar de mas o reiniciar sin necesidad. P_ceiling solo se usa aqui.
  4. El tiempo de espera NO lo decide el agente (fijo e igual para todos los metodos): se aisla lo que aportan beta y el reinicio.
  5. Episodios que contienen una busqueda inicial, un cambio abrupto de sombra (80 % de los episodios) y la re-busqueda.
Accion (Box [-1,1]^2):  a0 -> beta = 0.05 + 0.90*(a0+1)/2 ;  a1 > 0.8 -> reiniciar el enjambre (umbral alto para que la politica
inicial casi nunca reinicie y aprenda a hacerlo cuando conviene).
"""
import numpy as np
import gymnasium as gym
from gymnasium import spaces

from pvsim_V2.array_model_V2 import SyntheticPVCurve, random_shading_mix
from pvsim_V2.converter_V2 import Plant
from pvsim_V2.passrunner_V2 import PassRunner, FEATURES

N_PART = 7
RESET_THRESH = 0.8


def decode_action(a):
    beta = 0.05 + 0.90 * (float(np.clip(a[0], -1, 1)) + 1.0) / 2.0
    reset = bool(a[1] > RESET_THRESH)
    return beta, reset


class MPPTSwarmEnvV2(gym.Env):
    metadata = {"render_modes": []}

    def __init__(self, wait=5e-3, ep_len=130, p_change=0.8, change_range=(20, 70), n_sub=4,
                 obs_features=None, dt=1e-5):
        super().__init__()
        self.wait, self.ep_len, self.p_change, self.change_range = wait, ep_len, p_change, change_range
        self.n_sub, self.dt = n_sub, dt
        names = list(obs_features) if obs_features is not None else list(FEATURES)
        self.obs_idx = np.array([FEATURES.index(n) for n in names])
        self.action_space = spaces.Box(-1.0, 1.0, shape=(2,), dtype=np.float32)
        self.observation_space = spaces.Box(-10.0, 10.0, shape=(len(names),), dtype=np.float32)
        self._rng = np.random.default_rng()
        self.T_pass = N_PART * wait

    def _curve(self):
        return SyntheticPVCurve(random_shading_mix(self._rng, n_sub=self.n_sub))

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        if seed is not None:
            self._rng = np.random.default_rng(seed)
        options = options or {}
        self.curve = options.get("curve_a") or self._curve()
        self.curve_b = options.get("curve_b")
        if self.curve_b is None and self._rng.random() < self.p_change:
            self.curve_b = self._curve()
        self.change_at = options.get("change_at")
        if self.change_at is None:
            self.change_at = int(self._rng.integers(*self.change_range)) if self.curve_b is not None else None
        self.plant = Plant(self.curve, dt=self.dt)
        self.runner = PassRunner(self.wait)
        self.runner.start(self.plant)
        self.k = 0
        return self.runner.observe()[self.obs_idx], {}

    def step(self, action):
        if self.change_at is not None and self.k == self.change_at:   # cambio abrupto de sombra, sin aviso al agente
            self.curve = self.curve_b
            self.plant.set_curve(self.curve)
        beta, reset = decode_action(action)
        info = self.runner.do_pass(self.plant, beta, reset)
        reward = info["E"] / (self.curve.P_ceiling * self.T_pass)      # eficiencia de energia de esta pasada
        self.k += 1
        truncated = self.k >= self.ep_len
        info.update(reward_eff=reward, P_ceiling=self.curve.P_ceiling)
        return self.runner.observe()[self.obs_idx], float(reward), False, truncated, info
```

pvsim_V2/converter_V2.py

183 LÍNEAS

```
"""
converter_V2 — modelo dinamico PV + convertidor Boost + controlador Super-Twisting (ST-SMC),
ecuaciones (2)-(7) del paper, integrado con RK4 de paso fijo y COMPILADO con numba.

POR QUE ESTA REESCRITURA (respecto a pvsim/converter.py de la V1):
  La V1 integraba con scipy.solve_ivp (Python puro en cada evaluacion de la EDO): ~5 ms de computo por
  cada ms simulado. Con los tiempos fieles al paper (7 particulas x 0.5 ms, 25 pasadas, decenas de miles
  de episodios de entrenamiento) eso era inviable. Mismo modelo fisico, nucleo compilado => ~100x mas rapido.
  La equivalencia numerica con la V1 se verifica en validar_simulador_V2.py (resultado en la bitacora).
```

```
Estado x = [Vpv, Ipv, Vo, z]:
  Vpv: voltaje en el capacitor de entrada (terminales del panel)     Ipv: corriente del inductor
  Vo : voltaje de salida                                              z  : integrador del Super-Twisting
EDO (promediada, sin PWM):
  dVpv/dt = (I_panel(Vpv) - Ipv)/Ce          (balance de corriente en Ce; I_panel = curva I(V) del panel)
  dIpv/dt = Vpv/L - (1-u) Vo/L               (ec. 2)
  dVo/dt  = (1-u) Ipv/Cs - Vo/(R Cs)         (ec. 3)
Control u = u_eq + u_st  (ec. 4), con s = Ipv - I_ref (ec. 5), u_eq = 1 - Vpv/Vo (ec. 6),
  u_st = -lambda*sqrt(|s|)*sign(s) - z  (ec. 7),  dz/dt = Upsilon*sign(s);  u saturado a [0,1].
sign(s) se suaviza con tanh(s/eps) (el modelo promediado no resuelve el chattering de conmutacion).

La potencia que ve el algoritmo MPPT es la que REALMENTE entrega el panel: P = Vpv * I_panel(Vpv)
(no Vpv*Ipv: Ipv es la corriente del inductor y en transitorios difiere de la del panel; ver bitacora V1).
"""
import math
import numpy as np
from numba import njit

# Parametros del Boost (Tabla 2 del paper)
R_LOAD = 12.0
CS = 470e-6
L_IND = 10e-3
CE = 330e-6
# Ganancias del Super-Twisting tomadas del paper (ref. [43])
LAMBDA = 0.1
UPSILON = 0.01
SIGN_EPS = 1e-3


@njit(cache=True)
def _interp(x, xp, fp):
    """Interpolacion lineal escalar (xp ascendente); satura en los extremos."""
    n = xp.shape[0]
    if x <= xp[0]:
        return fp[0]
    if x >= xp[n - 1]:
        return fp[n - 1]
    lo = 0
    hi = n - 1
    while hi - lo > 1:
        mid = (lo + hi) // 2
        if xp[mid] <= x:
            lo = mid
        else:
            hi = mid
    dx = xp[hi] - xp[lo]
    if dx <= 0.0:
        return fp[lo]
    return fp[lo] + (x - xp[lo]) / dx * (fp[hi] - fp[lo])


@njit(cache=True)
def _rhs(s0, s1, s2, s3, Iref, Vx, Ix, prm):
    """Lado derecho de la EDO. prm = [L, Ce, Cs, R, lambda, upsilon, eps]."""
    L, Ce, Cs, R, lam, ups, eps = prm[0], prm[1], prm[2], prm[3], prm[4], prm[5], prm[6]
    Vo = s2 if s2 > 1e-3 else 1e-3
    s = s1 - Iref
    th = math.tanh(s / eps)
    ueq = 1.0 - s0 / Vo
    ust = -lam * math.sqrt(abs(s)) * th - s3
    u = ueq + ust
    if u < 0.0:
        u = 0.0
    elif u > 1.0:
        u = 1.0
    Ip = _interp(s0, Vx, Ix)
    return ((Ip - s1) / Ce,
            s0 / L - (1.0 - u) * Vo / L,
            (1.0 - u) * s1 / Cs - Vo / (R * Cs),
            ups * th)


@njit(cache=True)
def run_window(st, Iref, T, dt_max, Vx, Ix, prm):
    """Integra la EDO durante T segundos con I_ref constante (RK4, paso fijo <= dt_max).
    Modifica `st` en sitio. Devuelve (P_final, E): potencia del panel al final de la ventana (W) y
    energia entregada por el panel en la ventana (J)."""
    n = int(math.ceil(T / dt_max))
    if n < 1:
        n = 1
    h = T / n
    a0, a1, a2, a3 = st[0], st[1], st[2], st[3]
    P_prev = a0 * _interp(a0, Vx, Ix)
    E = 0.0
    for _ in range(n):
        k10, k11, k12, k13 = _rhs(a0, a1, a2, a3, Iref, Vx, Ix, prm)
        k20, k21, k22, k23 = _rhs(a0 + 0.5 * h * k10, a1 + 0.5 * h * k11, a2 + 0.5 * h * k12, a3 + 0.5 * h * k13, Iref, Vx, Ix, prm)
        k30, k31, k32, k33 = _rhs(a0 + 0.5 * h * k20, a1 + 0.5 * h * k21, a2 + 0.5 * h * k22, a3 + 0.5 * h * k23, Iref, Vx, Ix, prm)
        k40, k41, k42, k43 = _rhs(a0 + h * k30, a1 + h * k31, a2 + h * k32, a3 + h * k33, Iref, Vx, Ix, prm)
        a0 += h * (k10 + 2.0 * k20 + 2.0 * k30 + k40) / 6.0
        a1 += h * (k11 + 2.0 * k21 + 2.0 * k31 + k41) / 6.0
```

```
        a2 += h * (k12 + 2.0 * k22 + 2.0 * k32 + k42) / 6.0
        a3 += h * (k13 + 2.0 * k23 + 2.0 * k33 + k43) / 6.0
        if a0 < 0.0:
            a0 = 0.0
        P_new = a0 * _interp(a0, Vx, Ix)
        E += 0.5 * (P_prev + P_new) * h
        P_prev = P_new
    st[0], st[1], st[2], st[3] = a0, a1, a2, a3
    return P_prev, E


@njit(cache=True)
def run_window_trace(st, Iref, T, dt_max, Vx, Ix, prm, out_t, out_P, out_V, out_I):
    """Igual que run_window pero guarda la trayectoria (para graficas). Arreglos de tamano n+1."""
    n = int(math.ceil(T / dt_max))
    if n < 1:
        n = 1
    h = T / n
    a0, a1, a2, a3 = st[0], st[1], st[2], st[3]
    out_t[0] = 0.0
    out_P[0] = a0 * _interp(a0, Vx, Ix)
    out_V[0] = a0
    out_I[0] = a1
    for i in range(n):
        k10, k11, k12, k13 = _rhs(a0, a1, a2, a3, Iref, Vx, Ix, prm)
        k20, k21, k22, k23 = _rhs(a0 + 0.5 * h * k10, a1 + 0.5 * h * k11, a2 + 0.5 * h * k12, a3 + 0.5 * h * k13, Iref, Vx, Ix, prm)
        k30, k31, k32, k33 = _rhs(a0 + 0.5 * h * k20, a1 + 0.5 * h * k21, a2 + 0.5 * h * k22, a3 + 0.5 * h * k23, Iref, Vx, Ix, prm)
        k40, k41, k42, k43 = _rhs(a0 + h * k30, a1 + h * k31, a2 + h * k32, a3 + h * k33, Iref, Vx, Ix, prm)
        a0 += h * (k10 + 2.0 * k20 + 2.0 * k30 + k40) / 6.0
        a1 += h * (k11 + 2.0 * k21 + 2.0 * k31 + k41) / 6.0
        a2 += h * (k12 + 2.0 * k22 + 2.0 * k32 + k42) / 6.0
        a3 += h * (k13 + 2.0 * k23 + 2.0 * k33 + k43) / 6.0
        if a0 < 0.0:
            a0 = 0.0
        out_t[i + 1] = (i + 1) * h
        out_P[i + 1] = a0 * _interp(a0, Vx, Ix)
        out_V[i + 1] = a0
        out_I[i + 1] = a1
    st[0], st[1], st[2], st[3] = a0, a1, a2, a3
    return n


class Plant:
    """Planta PV + Boost + ST-SMC con estado persistente entre ventanas (la inercia del convertidor
    se arrastra de un candidato al siguiente, como en el hardware real)."""

    def __init__(self, curve, lam=LAMBDA, ups=UPSILON, eps=SIGN_EPS, dt=1e-5, R=R_LOAD):
        self.curve = curve
        self.prm = np.array([L_IND, CE, CS, R, lam, ups, eps], dtype=np.float64)
        self.dt = dt
        self.st = np.array([0.5, 0.0, 0.5, 0.0], dtype=np.float64)  # arranque en frio (igual que la V1)
        self.t = 0.0

    @property
    def Ipv(self):
        """Corriente de SALIDA DEL MODULO en los terminales del panel al final de la ultima ventana,
        I_panel(Vpv). Es la &#x27;Ipv' del paper (Eq.1: 'corriente de salida del modulo'); la usa MSAPSO en
        Gbest = Ipv y en Ppv = Vpv*Ipv. Asi Pbest y Gbest son siempre el MISMO punto de la curva P-V.
        (La corriente del inductor, st[1], difiere de esta durante los transitorios.)"""
        return float(np.interp(self.st[0], self.curve.Vx, self.curve.Ix))

    @property
    def I_inductor(self):
        return float(self.st[1])

    def set_curve(self, curve):
        """Cambio abrupto de sombra: cambia la fuente, el convertidor conserva su estado."""
        self.curve = curve

    def run(self, duration, I_ref):
        """Aplica I_ref durante `duration` s. Devuelve (P_final [W], E [J])."""
        P, E = run_window(self.st, float(I_ref), float(duration), self.dt, self.curve.Vx, self.curve.Ix, self.prm)
        self.t += duration
        return P, E

    def run_trace(self, duration, I_ref):
        n = int(math.ceil(duration / self.dt))
        t = np.empty(n + 1); P = np.empty(n + 1); V = np.empty(n + 1); I = np.empty(n + 1)
        run_window_trace(self.st, float(I_ref), float(duration), self.dt, self.curve.Vx, self.curve.Ix, self.prm, t, P, V, I)
        t = t + self.t
        self.t += duration
        return t, P, V, I
```

pvsim_V2/reinforce_V2.py 126 LÍNEAS

```
"""
reinforce_V2 — RAMA REINFORCE (gradiente de politica gaussiana) aplicada al MSAPSO, propuesta del profesor.

Idea: en vez de la actualizacion de MSAPSO x <- x + beta (Gbest - x), cada pasada se MUESTREAN K=7 corrientes candidatas de una
gaussiana N(mu, sigma^2) (en por unidad: corriente/Isc, asi sigma=0.3/0.5/0.8 son fracciones de Isc), se prueban en ESCALERA
(pasada ascendente, luego descendente, alternando; cada candidato se mantiene t_wait; Gbest se actualiza al instante), y se actualiza
mu con el gradiente del log de la gaussiana:  d/dmu log N(b; mu, sigma) = (b - mu)/sigma^2.
Parte «park» y regla de reinicio (|Pbest - Ppv| >= 5 W) IDENTICAS al MSAPSO fiel => se compara solo la regla de actualizacion.

Formas de la actualizacion (Delta_mu):
  form="B"  (forma del profesor):  lr_t * W * (G - mu) / sigma^2     con G = Gbest/Isc y W = R_G  (recompensa asociada a Gbest)
            (equivale a SAPSO con paso beta_eff = lr_t*W/sigma^2; el &#x27;baseline' resta a W: none | V)
  form="A"  (estimador REINFORCE de libro): lr_t * (1/K) * sum_k w_k (b_k - mu)/sigma^2, con w_k = R_k - base_k
Recompensa R_k = P_k / P_norm,  P_norm = potencia nominal 250 W ("rated") o la mejor potencia vista en esta busqueda ("best").
Baselines (para la forma A):
  none : w = R                          V : w = R - V (V = media de R de las muestras previas de esta busqueda)
  Q    : w = R - Q^(b)                  QV: w = Q^(b) - V    (Q^ = regresion kernel gaussiana de R sobre b con las muestras previas)
Tasas de aprendizaje lr_t:  "fixed" (0.1 | 0.01 | 0.001),  "accel" (Eq.14: 0.1 -> 0.95),  "inv_t" (1/t, t = pasada desde el reinicio).
Paso efectivo c = lr*W/sigma^2: converge si 0 < c < 2 (sin sobrepaso si c <= 1); se registra para el analisis de estabilidad.
"""
import numpy as np

P_RATED = 250.0
BW = 0.1          # ancho de banda (por unidad) del regresor kernel Q^


class ReinforceController:
    def __init__(self, wait, form="A", baseline="none", lr="fixed", lr_value=0.01, sigma=0.5, rnorm="best",
                 K=7, t_max=25, thresh=5.0, mu0=0.5, rng=None, beta_min=0.1, beta_max=0.95):
        self.wait, self.form, self.baseline, self.lr, self.lr_value = wait, form, baseline, lr, lr_value
        self.sigma, self.rnorm, self.K, self.t_max, self.thresh, self.mu0 = sigma, rnorm, K, t_max, thresh, mu0
        self.beta_min, self.beta_max = beta_min, beta_max
        self.rng = rng or np.random.default_rng(0)
        self.started = False
        self.mode = "search"
        self.n_restarts = 0
        self.sw = self  # compatibilidad con run_scenario (usa ctrl.sw.n)
        self.n = K

    # ---------------- estado de una busqueda ----------------
    def _restart(self, plant, first=False):
        self.Isc = float(plant.curve.Isc)
        self.mu = self.mu0
        self.Pbest = 0.0
        self.Gbest = 0.0                      # corriente (A) del mejor punto medido
        self.t = 1
        self.direction = "ascending"
        self.hist_b, self.hist_R = [], []
        self.c_last = 0.0
        if not first:
            self.n_restarts += 1

    def _lr_t(self):
        if self.lr == "fixed":
            return self.lr_value
        if self.lr == "accel":
            return self.beta_min + (self.t / self.t_max) * (self.beta_max - self.beta_min)
        if self.lr == "inv_t":
            return 1.0 / self.t
        raise ValueError(self.lr)

    def _qhat(self, b, V):
        if not self.hist_b:
            return V
        hb, hR = np.array(self.hist_b), np.array(self.hist_R)
        w = np.exp(-0.5 * ((b - hb) / BW) ** 2)
        s = w.sum()
        return float((w * hR).sum() / s) if s > 1e-9 else V

    # ---------------- un paso de tiempo ----------------
    def step(self, plant):
        if not self.started:
            self._restart(plant, first=True)
            self.started = True
        if self.mode == "park":
            P, E = plant.run(self.K * self.wait, self.Gbest)
            restarted = False
            if abs(self.Pbest - P) >= self.thresh:
                self._restart(plant); self.mode = "search"; restarted = True
            return dict(E=E, P_end=P, P_max=P, beta=0.0, mode="park", disp=self.sigma, t=self.t, Gbest=self.Gbest,
                        Pbest=self.Pbest, restart=restarted)

        b = np.clip(self.rng.normal(self.mu, self.sigma, self.K), 0.02, 1.0)        # candidatos en por unidad
        order = np.argsort(b)
        if self.direction == "descending":
            order = order[::-1]
        P = np.zeros(self.K); E = 0.0
        for i in order:                                                              # escalera
            Pi, e = plant.run(self.wait, b[i] * self.Isc)
```

```
            P[i] = Pi; E += e
            if Pi >= self.Pbest:                                        # Gbest inmediato (Ppv >= Pbest)
                self.Pbest, self.Gbest = Pi, plant.Ipv
        Pn = P_RATED if self.rnorm == "rated" else max(self.Pbest, 1e-6)
        R = P / Pn
        lr = self._lr_t()
        V = float(np.mean(self.hist_R)) if self.hist_R else float(R.mean())
        if self.form == "B":
            W = self.Pbest / Pn                                         # recompensa asociada a Gbest
            if self.baseline == "V":
                W = W - V
            G = self.Gbest / self.Isc
            c = lr * W / self.sigma ** 2
            d_mu = c * (G - self.mu)
        else:
            if self.baseline == "none":
                w = R
            elif self.baseline == "V":
                w = R - V
            elif self.baseline == "Q":
                w = R - np.array([self._qhat(bk, V) for bk in b])
            elif self.baseline == "QV":
                w = np.array([self._qhat(bk, V) for bk in b]) - V
            else:
                raise ValueError(self.baseline)
            d_mu = lr * float(np.mean(w * (b - self.mu))) / self.sigma ** 2
            c = d_mu / max(abs(float(np.mean(b)) - self.mu), 1e-9)
        self.hist_b.extend(b.tolist()); self.hist_R.extend(R.tolist())
        self.mu = float(np.clip(self.mu + d_mu, 0.02, 1.0))
        self.c_last = float(c)
        t_now = self.t
        self.t += 1
        self.direction = "descending" if self.direction == "ascending" else "ascending"
        if self.t > self.t_max:
            self.mode = "park"
        return dict(E=E, P_end=float(P[order[-1]]), P_max=float(P.max()), beta=float(lr), mode="search", disp=self.sigma,
                    t=t_now, Gbest=self.Gbest, Pbest=self.Pbest, restart=False, mu=self.mu, c=float(c))
```

pvsim_V2/reinforce_dist_V3.py

140 LÍNEAS

```
"""
reinforce_dist_V3 — Rama REINFORCE con DOS familias de politica: GAUSSIANA (la de la V2) y BETA (pedida por el profesor).

Todo lo demas es IDENTICO a pvsim_V2/reinforce_V2.py (K = 7 candidatos por pasada en por unidad b = I/Isc, escalera
ascendente/descendente alternada, Gbest inmediato, park tras t_max y reinicio si |Pbest - Ppv| >= 5 W, formas A y B,
baselines none | V | Q | QV, tasas fija | accel | inv_t). Solo cambian (1) como se muestrean los candidatos y
(2) la funcion de puntaje (score) d/dmu log pi(b | mu) que usa el gradiente de politica.

GAUSSIANA  b ~ N(mu, sigma^2), recortada a [0.02, 1]          score(b) = (b - mu) / sigma^2
           (se conserva EXACTAMENTE el calculo de la V2: el score se evalua en la muestra ya recortada)

BETA       b ~ Beta(a, c) con a = mu*kappa, c = (1 - mu)*kappa   (parametrizacion media-concentracion; soporte (0, 1))
           score(b) = d/dmu log Beta(b; mu*kappa, (1-mu)*kappa)
                    = kappa * [ ln b - ln(1 - b) - digamma(a) + digamma(c) ]
           La corriente APLICADA se recorta a [0.02, 1] igual que en la gaussiana (mismo limite fisico), pero el score
           se evalua en la muestra SIN recortar, que es la que realmente salio de la politica (estimador sin sesgo).

Comparacion justa: la dispersion se fija con el mismo parametro "sigma" para ambas. Para la Beta se elige kappa de modo
que su desviacion estandar en mu = 0.5 sea igual a sigma:  Var = mu(1-mu)/(kappa+1)  =>  kappa = 0.25/sigma^2 - 1.
Consecuencias que hay que tener presentes al interpretar:
  * Una Beta no puede tener desviacion >= 0.5, asi que sigma solo puede ser < 0.5 (la V2 probo 0.5 y 0.8 en la gaussiana).
  * Con sigma = 0.3 -> kappa = 1.78: en mu = 0.5, a = c = 0.89 < 1 y la densidad es en forma de U (masa en los extremos).
  * Con kappa fijo, la desviacion de la Beta se encoge sola cuando mu se acerca a 0 o a 1 (la gaussiana no).
Para la forma B (la del profesor) el "punto objetivo" es G = Gbest/Isc; con la Beta se recorta G a [0.02, 0.98] para que
ln(1 - G) sea finito.
"""
import numpy as np
from scipy.special import digamma
from pvsim_V2.reinforce_V2 import ReinforceController, P_RATED

B_MIN, B_MAX = 0.02, 1.0        # rango fisico aplicado (igual que la V2)
MU_MAX_BETA = 0.98              # la media de una Beta debe quedar estrictamente dentro de (0, 1)
EPS = 1e-6


def kappa_from_sigma(sigma):
    """Concentracion de la Beta con la misma desviacion que la gaussiana en mu = 0.5."""
    k = 0.25 / sigma ** 2 - 1.0
    if k <= 0:
        raise ValueError(f"sigma = {sigma} no es alcanzable por una Beta (debe ser < 0.5)")
    return k


class ReinforceDistController(ReinforceController):
    """REINFORCE con politica &#x27;gauss' (identica a la V2) o 'beta'."""
```

```
    def __init__(self, wait, dist="gauss", **kw):
        super().__init__(wait, **kw)
        self.dist = dist
        if dist == "beta":
            self.kappa = kappa_from_sigma(self.sigma)
        elif dist != "gauss":
            raise ValueError(dist)

    # ---------- politica ----------
    def _mu_max(self):
        return MU_MAX_BETA if self.dist == "beta" else 1.0

    def _sample(self):
        """Devuelve (b_aplicada, b_para_score)."""
        if self.dist == "gauss":
            b = np.clip(self.rng.normal(self.mu, self.sigma, self.K), B_MIN, B_MAX)
            return b, b                                  # V2: score sobre la muestra recortada
        a, c = self.mu * self.kappa, (1.0 - self.mu) * self.kappa
        raw = np.clip(self.rng.beta(a, c, self.K), EPS, 1.0 - EPS)
        return np.clip(raw, B_MIN, B_MAX), raw

    def _score(self, b):
        b = np.asarray(b, dtype=float)
        if self.dist == "gauss":
            return (b - self.mu) / self.sigma ** 2
        a, c = self.mu * self.kappa, (1.0 - self.mu) * self.kappa
        b = np.clip(b, EPS, 1.0 - EPS)
        return self.kappa * (np.log(b) - np.log1p(-b) - digamma(a) + digamma(c))

    def spread(self):
        """Desviacion estandar actual de la politica (por unidad)."""
        if self.dist == "gauss":
            return self.sigma
        return float(np.sqrt(self.mu * (1 - self.mu) / (self.kappa + 1)))

    # ---------- un paso de tiempo (misma estructura que reinforce_V2.step) ----------
    def step(self, plant):
        if not self.started:
            self._restart(plant, first=True)
            self.started = True
        if self.mode == "park":
            P, E = plant.run(self.K * self.wait, self.Gbest)
            restarted = False
            if abs(self.Pbest - P) >= self.thresh:
                self._restart(plant); self.mode = "search"; restarted = True
            return dict(E=E, P_end=P, P_max=P, beta=0.0, mode="park", disp=self.spread(), t=self.t, Gbest=self.Gbest,
                        Pbest=self.Pbest, restart=restarted, mu=self.mu)

        b, b_score = self._sample()
        order = np.argsort(b)
        if self.direction == "descending":
            order = order[::-1]
        P = np.zeros(self.K); E = 0.0
        for i in order:                                                  # escalera
            Pi, e = plant.run(self.wait, b[i] * self.Isc)
            P[i] = Pi; E += e
            if Pi >= self.Pbest:                                         # Gbest inmediato (Ppv >= Pbest)
                self.Pbest, self.Gbest = Pi, plant.Ipv
        Pn = P_RATED if self.rnorm == "rated" else max(self.Pbest, 1e-6)
        R = P / Pn
        lr = self._lr_t()
        V = float(np.mean(self.hist_R)) if self.hist_R else float(R.mean())
        if self.form == "B":
            W = self.Pbest / Pn
            if self.baseline == "V":
                W = W - V
            G = self.Gbest / self.Isc
            if self.dist == "beta":
                G = float(np.clip(G, B_MIN, MU_MAX_BETA))
            d_mu = lr * W * float(self._score(G))
            c = d_mu / (G - self.mu) if abs(G - self.mu) > 1e-9 else 0.0
        else:
            if self.baseline == "none":
                w = R
            elif self.baseline == "V":
                w = R - V
            elif self.baseline == "Q":
                w = R - np.array([self._qhat(bk, V) for bk in b])
            elif self.baseline == "QV":
                w = np.array([self._qhat(bk, V) for bk in b]) - V
            else:
                raise ValueError(self.baseline)
            d_mu = lr * float(np.mean(w * self._score(b_score)))
            c = d_mu / max(abs(float(np.mean(b)) - self.mu), 1e-9)
        self.hist_b.extend(b.tolist()); self.hist_R.extend(R.tolist())
        self.mu = float(np.clip(self.mu + d_mu, B_MIN, self._mu_max()))
        self.c_last = float(c)
        t_now = self.t
        self.t += 1
        self.direction = "descending" if self.direction == "ascending" else "ascending"
```

```
        if self.t > self.t_max:
            self.mode = "park"
        return dict(E=E, P_end=float(P[order[-1]]), P_max=float(P.max()), beta=float(lr), mode="search", disp=self.spread(),
                    t=t_now, Gbest=self.Gbest, Pbest=self.Pbest, restart=False, mu=self.mu, c=float(c))
```

train_V2.py

61 LÍNEAS

```
"""
train_V2.py — Entrenamiento PPO (stable-baselines3) del agente que decide beta y reinicio en MPPTSwarmEnvV2.

USO:
  python3 train_V2.py --wait 5e-3 --seed 0 --timesteps 1000000 --n-envs 4 --tag main_s0
  python3 train_V2.py --wait 5e-3 --seed 0 --obs full|no_disp|no_time|no_drop|minimal ... (ablacion de senales)
Hiperparametros de PPO: los POR DEFECTO de stable-baselines3 (n_steps=2048, batch=64, gamma=0.99, gae_lambda=0.95,
clip=0.2, lr=3e-4), red 64x64 para politica y valor. Solo cambia el numero de pasos, el # de entornos y la semilla.
"""
import argparse, os, sys, json, time
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import numpy as np
from stable_baselines3 import PPO, SAC, A2C
from stable_baselines3.common.vec_env import SubprocVecEnv, DummyVecEnv, VecMonitor
from stable_baselines3.common.callbacks import CheckpointCallback
from pvsim_V2.gym_env_V2 import MPPTSwarmEnvV2
from pvsim_V2.passrunner_V2 import FEATURES

OBS_VARIANTS = {
    "full":     FEATURES,
    "no_disp":  [f for f in FEATURES if f != "dispersion"],
    "no_time":  [f for f in FEATURES if f != "t_reinicio"],
    "no_drop":  [f for f in FEATURES if f not in ("caida_rel", "dPbest")],      # sin las senales explicitas de caida/subida
    "minimal":  ["P_max", "Pbest", "caida_rel"],
}


def make_env(wait, feats, seed):
    def _init():
        env = MPPTSwarmEnvV2(wait=wait, obs_features=feats)
        env.reset(seed=seed)
        return env
    return _init


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--wait", type=float, default=5e-3)
    ap.add_argument("--seed", type=int, default=0)
    ap.add_argument("--timesteps", type=int, default=1_000_000)
    ap.add_argument("--n-envs", type=int, default=4)
    ap.add_argument("--obs", default="full", choices=list(OBS_VARIANTS))
    ap.add_argument("--algo", default="ppo", choices=["ppo", "sac", "a2c"])
    ap.add_argument("--tag", default=None)
    ap.add_argument("--outdir", default="modelos_V2")
    a = ap.parse_args()
    tag = a.tag or f"{a.algo}_w{a.wait*1e3:g}ms_{a.obs}_s{a.seed}"
    os.makedirs(a.outdir, exist_ok=True)
    feats = OBS_VARIANTS[a.obs]
    Vec = DummyVecEnv if a.n_envs == 1 else SubprocVecEnv
    venv = VecMonitor(Vec([make_env(a.wait, feats, 1000 * a.seed + i) for i in range(a.n_envs)]))
    Algo = {"ppo": PPO, "sac": SAC, "a2c": A2C}[a.algo]       # hiperparametros por defecto de stable-baselines3 en los tres
    model = Algo("MlpPolicy", venv, verbose=0, seed=a.seed, policy_kwargs=dict(net_arch=[64, 64]),
                 tensorboard_log=os.path.join(a.outdir, "tb"))
    ck = CheckpointCallback(save_freq=max(a.timesteps // 4 // a.n_envs, 1000), save_path=a.outdir, name_prefix=tag + "_ckpt")
    t0 = time.time()
    model.learn(total_timesteps=a.timesteps, callback=ck, tb_log_name=tag, progress_bar=False)
    model.save(os.path.join(a.outdir, tag))
    json.dump(dict(tag=tag, algo=a.algo, wait=a.wait, seed=a.seed, obs=a.obs, features=feats, timesteps=a.timesteps,
                   n_envs=a.n_envs, seconds=time.time() - t0), open(os.path.join(a.outdir, tag + ".json"), "w"), indent=1)
    print(f"{tag}: listo en {time.time()-t0:.0f} s")
```